User Manual

for S32K1_S32M24X LIN_43_LPUART_FLEXIO Driver

Document Number: UM2LIN_43_LPUART_FLEXIOASRR21-11 Rev0000R3.0.0 Rev. 1.0

Chapter 1

Revision History

Revision	Date	Author	Description
1.0	07.03.2025	NXP RTD Team	S32K1_S32M24X Real-Time Drivers AUTOSAR R21-11 Version 3.0.0

Chapter 2

Introduction

This User Manual describes NXP Semiconductor AUTOSAR LIN for S32K1_S32M24X product series. AUTOSAR LIN driver configuration parameters and deviations from the specification are described in Driver chapter of this document. AUTOSAR LIN driver requirements and APIs are described in the AUTOSAR LIN driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32k116_qfn32
- s32k116_lqfp48
- s32k118_lqfp48
- s32k118_lqfp64
- s32k142_lqfp48
- s32k142_lqfp64
- s32k142_lqfp100
- s32k142w_lqfp48
- s32k142w_lqfp64
- s32k144_lqfp48
- s32k144_lqfp64 / MWCT1014S_lqfp64
- s32k144_lqfp100 / MWCT1014S_lqfp100
- s32k144_mapbga100
- s32k144w_lqfp48
- s32k144w_lqfp64

- s32k146_lqfp64
- s32k146_lqfp100 / MWCT1015S_lqfp100
- s32k146_mapbga100 / MWCT1015S_mapbga100
- s32k146_lqfp144
- s32k148_lqfp100
- s32k148_mapbga100 / MWCT1016S_mapbga100
- s32k148_lqfp144
- s32k148_lqfp176
- s32m241_lqfp64
- s32m242_lqfp64
- s32m243_lqfp64
- s32m244_lqfp64

All of the above microcontroller devices are collectively named as S32K1_S32M24X. Note: MWCT part numbers contain NXP confidential IP for Qi Wireless Power

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
ASM	Assembler
BSMI	Basic Software Make file Interface
C/CPP	C and C++ Source Code
DEM	Diagnostic Event Manager
DET	Development Error Tracer
ECUM	ECU state Manager
ECU	Electronic Control Unit
ISR	Interrupt Service Routine
LIN	Local Interconnect Network
LSB	Least Signifigant Bit
MCU	Micro Controller Unit
MSB	Most Significant Bit
N/A	Not Applicable
RAM	Random Access Memory
SIU	Systems Integration Unit
SWS	Software Specification
VLE	Variable Length Encoding
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	Specification of Driver	AUTOSAR Release R21-11
2	S32K1xx Reference Manual	Rev.14.1, 01/2024
3	S32M24x Reference Manual	Rev. 5, 12/2024
4	S32K1xx Data Sheet	Rev. 14, 08/2021
5	S32M2xx Data Sheet	Rev. 7, 12/2024
6	S32K116 Errata	Mask Set Errata for Mask 0N96V Rev. 17/JAN/2024, 1/2024
7	S32K118 Errata	Mask Set Errata for Mask 0N97V Rev. 15/JAN/2024, 1/2024
8	S32K142 Errata	Mask Set Errata for Mask 0N33V Rev. 15/JAN/2024, 1/2024
9	S32K144 Errata	Mask Set Errata for Mask 0N57U Rev. 15/JAN/2024, 1/2024
10	S32K144W Errata	Mask Set Errata for Mask 0P64A Rev. 03/JUL/2024, 7/2024
11	S32K146 Errata	Mask Set Errata for Mask 0N73V Rev. 15/JAN/2024, 1/2024
12	S32K148 Errata	Mask Set Errata for Mask 0N20V Rev. 15/JAN/2024, 1/2024
13	S32M242 Errata	Mask Set Errata for Mask N33V+P69K Rev.1, 8/2024
14	S32M244 Errata	Mask Set Errata for Mask P64A+P69K Rev.1, 8/2024

Chapter 3

Driver

3.1 Requirements

Requirements for this driver are detailed in the Autosar Driver Software Specification document (See Table Reference List).

3.2 Driver Design Summary

The LIN driver is part of the Real Time Drivers, performs the hardware access and offers a hardware independent API to the upper layer.

The only upper layer, which has access to the LIN driver, is the LIN Interface.

A LIN driver can support more than one channel over LPUART and FLEXIO hardware units.

This LIN driver supports both MASTER and SLAVE mode over all the hardware units.

The LIN Driver for S32K1_S32M24X uses the LPUART and FLEXIO on-chip hardware modules which provides special support for the LIN protocol.

It can be used to automate most tasks of a LIN master and slave node.

It is possible to transmit entire frames (or sequences of frames) and receive data from LIN slaves.

The LIN physical interface should be connected to the LPUART module pins in order to get the LIN bus voltage levels. The same requirement applies also for FLEXIO module.

The LPUART hardware unit has the following major features:

- Transmit and receive baud rate can operate asynchronous to the bus clock

- Programmable baud rates (13-bit modulo divider) with configurable oversampling ratio from 4x to 32x
- Receive data register full, transmit data register empty and transmission complete interrupts
- Receive overrun, framing error, and noise error detection
- Optional 13-bit break character generation

The FLEXIO hardware unit has the following major features:

- Array of 32-bit shift registers with transmit, receive and data match modes
- Double buffered shifter operation for continuous data transfer
- Shifter concatenation to support large transfer sizes
- Automatic start/stop bit generation
- Programmable baud rates independent of bus clock frequency
- Interrupt transmit/receive operation
- Integrated general purpose input/output registers and pin rising/falling edge interrupts to simplify software support

3.3 Hardware Resources

The device family includes S32K116, S32K118, S32K142, S32K142W, S32K144, S32K144W, S32K146, S32K148, S32M24x derivatives.

The hardware modules configured by the Lin driver are LPUART and FLEXIO.

3.3.1 Low Power Universal Asynchronous Receiver/Transmitter (LPUART):

The LPUART Module Hardware Instances on the S32K1 family derivatives are listed in the following table.

Chip	Instances
S32K116	LPUART0
	LPUART1
S32K118	LPUART0
	LPUART1
S32K142	LPUART0
	LPUART1
S32K144	LPUART0
	LPUART1
	LPUART2
S32K144W	LPUART0
	LPUART1
	LPUART2
S32K146	LPUART0
	LPUART1
	LPUART2
S32K148	LPUART0
	LPUART1
	LPUART2

Figure 3.1 LPUART configuration in Reference manual

The LPUART Module Hardware Instances on the S32M24X family derivatives are listed in the following table.

Table 406. LPUART instances and features

Chip	Instances	TX FIFO (word)	RX FIFO (word)
S32M24x	LPUART0	4	4
	LPUART1	4	4

Figure 3.2 LPUART configuration in Reference manual

In order to configure a LPUART or FLEXIO LIN channel the following steps must be followed:

3.3.2 Flexible I/O (FlexIO):

In case of the FLEXIO module there is only one hardware instance on the chip. The FLEXIO_0, FLEXIO_1 are an abstractization of the resources that the FLEXIO unit uses in order to implement the Lin protocol. For more details about the FLEXIO hardware unit, please read "Flexible I/O (FlexIO)" subchapter of the Communication chapter of the Reference Manual.

3.3.3 Autosar configuration:

Go to Mcu component and enable the LPUART or FLEXIO Clock which is used by the desired Lpuart hardware channel. Create a refference to this clock.

Go to Lin component in your configurator. Select the desired Lpuart hardware channel. Use the Mcu refference previously configured.

In case of the FLEXIO module there is only one hardware instance on the chip. The FLEXIO_IP_0, FLEXIO_IP_1 are an abstractization of the resources that the FLEXIO unit uses in order to implement the Lin protocol. For more details about the FLEXIO hardware unit, please read "Flexible I/O (FlexIO)" subchapter of the Communication chapter of the Reference Manual.

Go to Platform component and enable the Lpuart hardware channel interrupt and configure the ISR Handler.

Go to Port component and create two Pins in the Port Container. Those pins must be configured as RX and TX, specific for the hardware instance used.

For example:

In EB tresos and also Design Studio, *LPUART_IP_0* channel has named *LPUART_IP_0*, correspondingly as below:

Figure 3.3 Lin hardware channel configuration in EB Tresos.

The peripheral clocks which must be enabled are LPUARTX_CLK where X is the number of hw instance used and FlexIO0_CLK.

In order to configure the pins, the files from Reference Manual shall be interogated in order to determine the PCR value of the required pin.

The LIN channel to S32K116 pin mapping can be done using file *S32K116_IO_Signal_Description_Input_↩ Multiplexing.xlsx* from Reference manual.

The LIN channel to S32K118 pin mapping can be done using file *S32K118_IO_Signal_Description_Input_↩ Multiplexing.xlsx* from Reference manual.

The LIN channel to S32K142 pin mapping can be done using file *S32K142_IO_Signal_Description_Input_↩ Multiplexing.xlsx* from Reference manual.

The LIN channel to S32K142W pin mapping can be done using file *S32K142W_IO_Signal_Description_Input_↩ Multiplexing.xlsx* from Reference manual.

The LIN channel to S32K144 pin mapping can be done using file *S32K144_IO_Signal_Description_Input_↩ Multiplexing.xlsx* from Reference manual.

The LIN channel to S32K144W pin mapping can be done using file *S32K144W_IO_Signal_Description_Input_↩ Multiplexing.xlsx* from Reference manual.

The LIN channel to S32K146 pin mapping can be done using file *S32K146_IO_Signal_Description_Input_↩ Multiplexing.xlsx* from Reference manual.

The LIN channel to S32K148 pin mapping can be done using file *S32K148_IO_Signal_Description_Input_Multiplexing.xlsx* from Reference manual.

The LIN channel to S32M24x pin mapping can be done using file *S32M24x_IOMUX.xlsx* from Reference manual.

3.4 Deviations from Requirements

The driver deviates from the AUTOSAR LIN Driver software specification in some places. The table below identifies the AUTOSAR requirements that are not implemented, not fully implemented or out of scope for the LIN Driver.

Term	Definition
N/S	Out of scope
N/I	Not implemented
N/F	Not fully implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, not implemented or out of scope for the LIN driver.

Requirement	Status	Description	Notes
SWS_Lin_00055	N/S	The Lin module shall fulfill all design and implementation guidelines as described in Specification of C Implementation Rules AUTOSAR_TR_CImplementationRules.pdf.	Requirement already covered by process.
SWS_Lin_00026	N/S	If the LIN hardware unit cannot queue the bytes for transmission or reception (e.g. simple UART implementation), the LIN driver shall provide a temporary communication buffer.	The LIN hardware already has a data buffer built-in.
SWS_Lin_00039	N/S	Values that can be configured are hardware dependent. Therefore, the rules and constraints cannot be given in the standard.	This is not a requirement.

Requirement	Status	Description	Notes
SWS_Lin_00999	N/S	These requirements are not applicable to this specification. (SRS_BSW_00307, SRS_BSW_00312, SRS_BSW_00325, SRS_BSW_00326, SRS_BSW_00328, SRS_BSW_00329, SRS_BSW_00330, SRS_BSW_00331, SRS_BSW_00336, SRS_BSW_00339, SRS_BSW_00342, SRS_BSW_00343, SRS_BSW_00353, SRS_BSW_00357, SRS_BSW_00359, SRS_BSW_00360, SRS_BSW_00361, SRS_BSW_00373, SRS_BSW_00376, SRS_BSW_00378, SRS_BSW_00383, SRS_BSW_00395, SRS_BSW_00397, SRS_BSW_00398, SRS_BSW_00399, SRS_BSW_00400, SRS_BSW_00413, SRS_BSW_00415, SRS_BSW_00416, SRS_BSW_00417, BSW00420, SRS_BSW_00422, SRS_BSW_00423, SRS_BSW_00424, SRS_BSW_00425, SRS_BSW_00426, SRS_BSW_00427, SRS_BSW_00428, SRS_BSW_00429, BSW00431, SRS_BSW_00432, SRS_BSW_00433, BSW00434, SRS_BSW_00005, SRS_BSW_00007, SRS_BSW_00162, SRS_BSW_00168, SRS_SPAL_12056, SRS_SPAL_12267, SRS_SPAL_12163, SRS_SPAL_12463, SRS_SPAL_12075, SRS_SPAL_12078, SRS_SPAL_12092, SRS_Lin_01551, SRS_Lin_01568, SRS_Lin_01569, SRS_Lin_01570, SRS_Lin_01564, SRS_Lin_01546, SRS_Lin_01561, SRS_Lin_01549, SRS_Lin_01571, SRS_Lin_01514, SRS_Lin_01515, SRS_Lin_01502, SRS_Lin_01558, BSW01527, SRS_Lin_01523, SRS_Lin_01540, SRS_Lin_01545, SRS_Lin_01534, SRS_Lin_01574, SRS_Lin_01539, SRS_Lin_01544, SRS_Lin_01590).	This is not a requirement.
SWS_Lin_00201	N/S	For different LIN hardware units a separate LIN driver needs to be implemented. It is up to the implementer to adapt the driver to the different instances of similar LIN channels.	Rejection reason: LIN driver is designed to adapt to different LIN hardware units since.

Requirement	Status	Description	Notes
SWS_Lin_00099	N/S	If development error detection for the Lin module is enabled: the function Lin_Init shall check the parameter Config for being within the allowed range. If Config is not in the allowed range, the function Lin_Init shall raise the development error LIN_E_INVA↩ LID_POINTER.	Requirement is marked Rejected, as it is replaced by CPR_RTD_00255

3.5 Driver Limitations

The limitations of this driver are:

- In Slave mode, Lin over Flexio can't distinguish frame error from overrun error, so LIN_ERR_INC_RESP will be reported to LinIf instead of LIN_ERR_RESP_STOPBIT
- In Master mode, the break byte length is not configurable. The only length supported is 13 bits long.
- Break length detection is not configurable (we only support 11-bits break length detect).
- Autobaud feature is not implemented over FLEXIO module.
- When LinFrameTimeoutDisable is ON, Short response and No response errors shall not supported by Lin driver.
- Lin driver doesn't support raise LIN_TX_HEADER_ERROR and LIN_TX_ERROR when physcial bus error occurs, because the hardware doesn't support detect this error. If TX_BUSY occurs for a long enough period of time (e.g. LinHeaderTimeout for header transmission and LinResponseTimeout for response transmission), user should consider it a bus error and recheck the bus.

3.6 Driver usage and configuration tips

3.6.1 Dual Clock Feature

Lin driver allows dynamic change of working frequency. The LinClockRef_Alternate node can be found at Lin/↩ LinGlobalConfig/[Configuration_name]/LinChannel/General. This node should be enabled in order to have this feature active. The frequency can be changed if Lin_SetClockMode function is call, after calling Lin_Init function.

Note

Our recommended usage for this API is to call it when the driver is in a lower power state but still in active use.

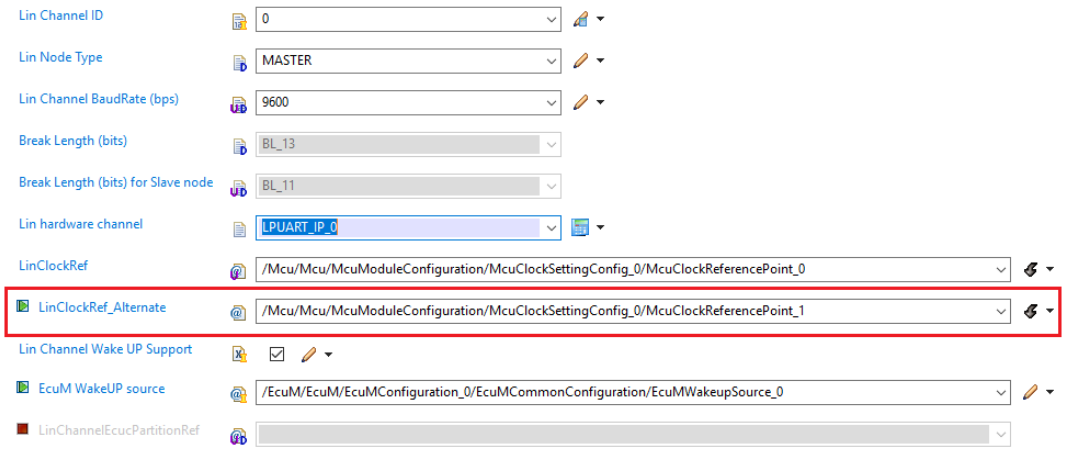

Figure 3.4 Dual Clock Feature configuration node in EB Tresos.

3.6.2 How to configure to detect the wake-up signal

To use the functions related to wakeup feature, "Detect the wake-up signal" must be configured. Because wake-up signal detection is not support by LPUART hardware, so driver will use the timer and Lpuart edge detection interrupts to detect and measure the wake up signal. The wake-up signal feature uses a timer which must be initialized in the application/test cases. Lin driver uses a notification functions which are intended to start and stop the measurement of the timeout period. These functions must be implemented by user. The notification will be called by interrupt of Lpuart when Lpuart detects both falling and rising edge of wake up signal. User need to calculate time get from twice of wake-up and change it to nanosecond. Driver only handles the nanosecond.

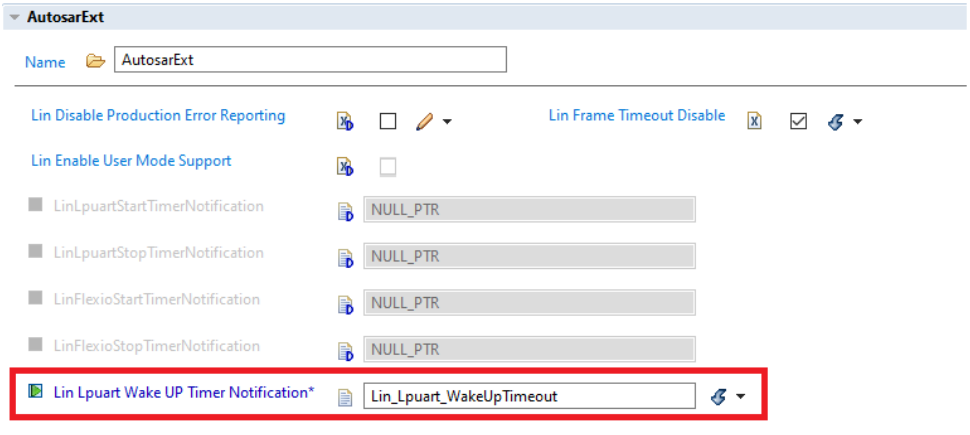

Figure 3.5 Callback function configuration.

3.6.3 Frame Timeout Disable Feature

In Autosar mode, the 'Lin Frame Timeout Disable' checkbox in AutosarExt container should be enabled in order to have this feature active. If LinFrameTimeoutDisable is ON then Lin driver will accept the frame that is longer than Maximal Frame Length. For the LIN driver, timeout feature is applied to both LIN 2.1 master and slave nodes, in response frame reception.

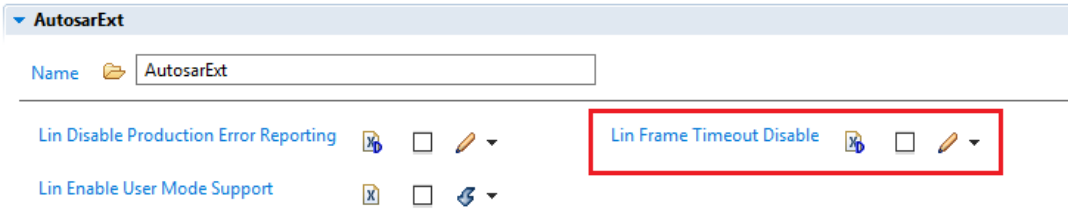

Figure 3.6 Frame Timeout Disable Feature configuration node in EB Tresos.

The frame timeout feature uses a timer which must be initialized in the application/test cases. Lin driver uses two notification functions which are intended to start and stop the measurement of the timeout period. These functions must be implemented by user.

The start function has the following prototype : void func (uint8 Channel, uint32 MicroSeconds); This function must start the timer to count the period of time provided via the second parameter.

The stop function has the following prototype : void func (uint8 Channel); This function must stop the timer counting.

Note: User should configure timeout notification function in timer IRQ handler, which must be called when timeout occurs via GptNotification node of GPT module.

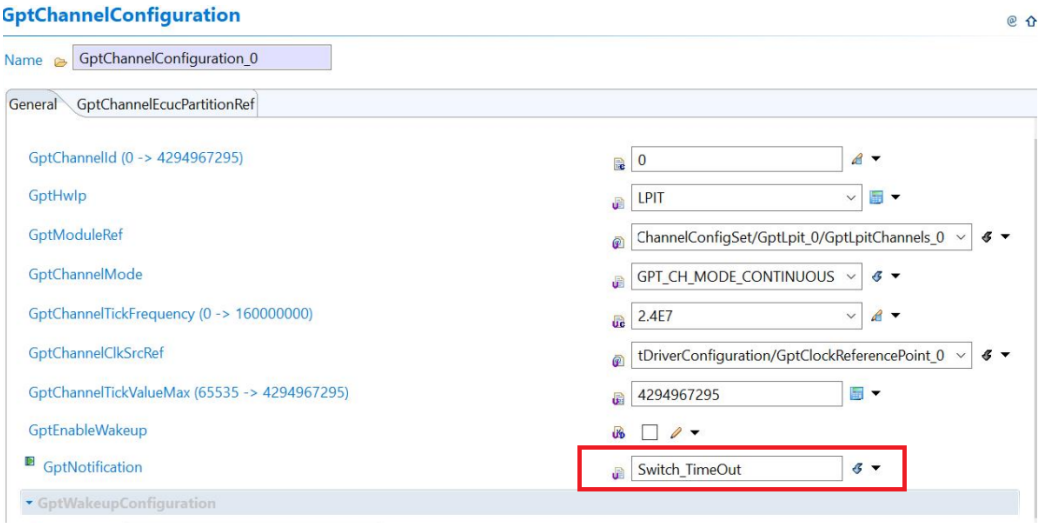

Figure 3.7 Timeout notification function configured in EB Tresos.

Switch_TimeOut, as shown in the diagram above, is a user-written timeout notification function that calls the Lin driver's corresponding Lpuart_Lin_Ip_TimerExpiredService or Flexio_Lin_Ip_TimerExpiredService functions when using Lpuart or Flexio.

If this feature is enabled, LinFrameTimeoutDisable is OFF (not scored-out in the configurators). The two notification functions must be also provided in the configurator. In the next figure, there is an example of a configuration in Design Studio for a Lpuart channel. In case of a Flexio channel, notification functions must be added separately in the Flexio specific fields.

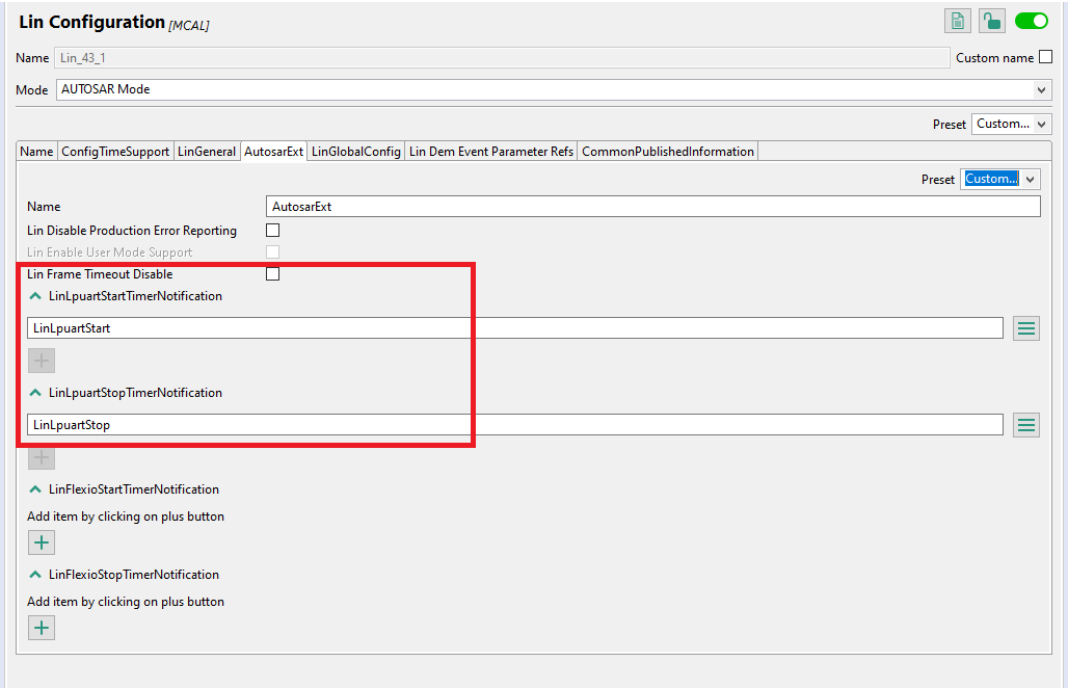

Figure 3.8 Frame Timeout Enabled Feature and notification functions configured in Design Studio.

3.6.4 Auto-detect baudrate feature

Autobaud is an extensive feature in Lpuart Lin Driver which allows a slave node to automatically detect baudrate of LIN bus and adapt its original baudrate to bus value. Auto Baud is applied when the baudrate of the incoming data is unknown.

This feature is only available for Lpuart Lin Ip driver.

This feature needs one Pin and one Timer. For Pin setting, we have 2 ways to implement:

- First of all (recommended), user uses Rx pin of current Lpuart channel, initializes and sets up it as GPIO mode with interrupt enable on both falling and rising edge. After autobaud process has finished successfully, user need to set up this Rx pin as Uart Rx mode.

- The other one, user can use any Pin, set up it as GPIO mode with interrupt enable on both falling and rising edge (or can set up output triger timer pin E.g FTM pin) and physically wires this pin to Rx pin. For Timer: This feature uses a timer (ex: FTM) in Input Capture Mode. The timer must be initialized in the application/test cases.

In order to use this feature, the Autobaud Feature must be enable in the configurator and a function must be provided and implemented.

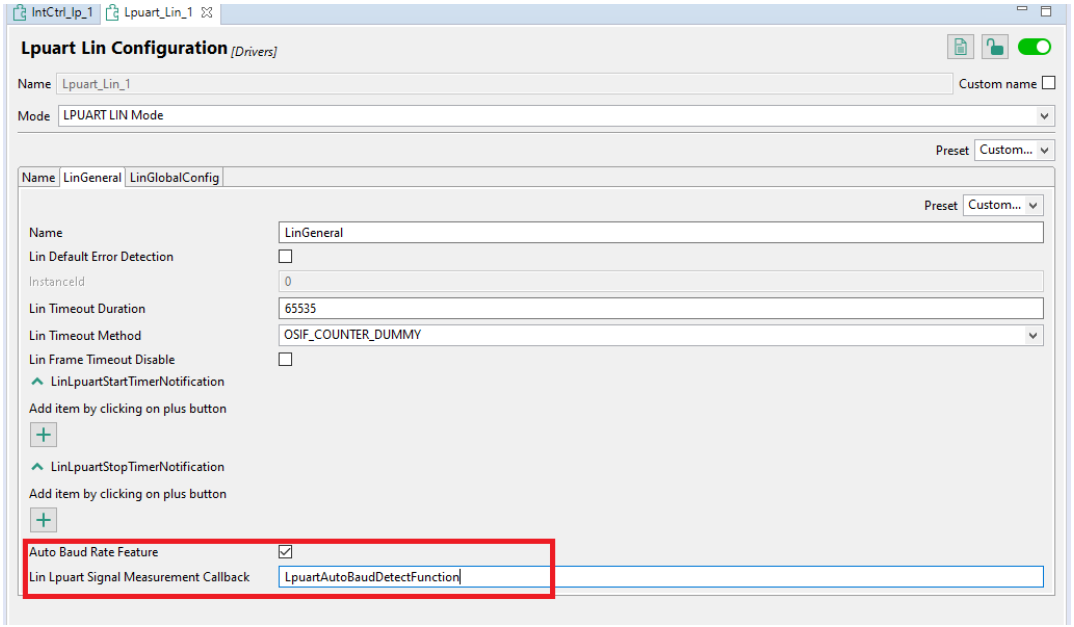

Figure 3.9 Autobaud Feature and notification functions configured in Design Studio

Enable Auto baud rate in LinChannel container:

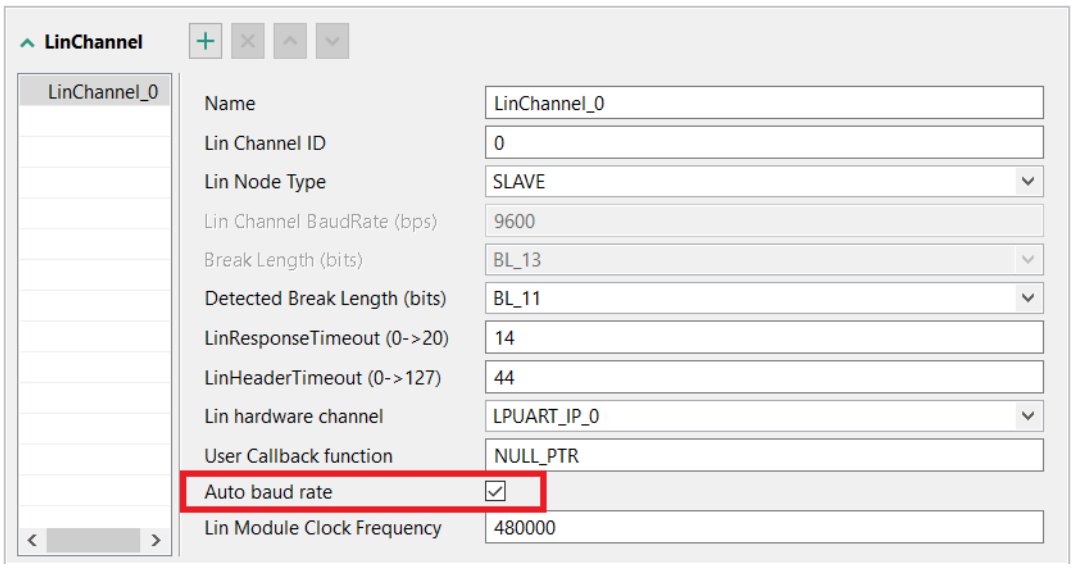

Figure 3.10 Autobaud Feature configured for channel in Design Studio

Users shall assign measurement callback function pointer in *Lin Lpuart Signal Measurement Callback* field in S32↩ DS. In the application/test cases, the function has the following prototype: "void func(uint8 Instance, uint32 *↩ Nanoseconds);". This function must assign into Nanoseconds parameter time period between two consecutive pin active edges in nano seconds. If this function is called for the first time, it will start the timer to measure time. When an event (such as detecting a falling edge of a dominant signal while node is in sleep mode) occurs, LIN driver will call this callback to start time measurement. Then on rising edge of that signal, LIN driver will call this callback function to get time interval of that dominant signal in nano seconds. If Autobaud feature is enabled, LIN driver uses this callback to measure two bit time length between two consecutive falling edges of the sync byte in order to evaluate Master's baudrate. Users can implement this function in their applications.

The application should use a Port Pin interrupt or (external pin trigger timer interrupt of both rising and falling edges(E.g FTM)), call Lpuart_Lin_Ip_AutoBaudCapture(uint8 Instance) function to calculate and set Slave's baudrate like Master's baudrate. When receiving a frame header, the slave detect LIN bus's baudrate based on the synchronization byte and adapts its baudrate accordingly. On changing baudrate, the slave set current event ID to LPUART_LIN_IP_BAUDRATE_ADJUSTED and call the callback function. In that callback function users might change the pin mode as Uart Rx mode.

Note: Baudrate evaluation process is executed until autobaud successfully, do not call any function else during this process.

3.6.5 How to configure a FLEXIO channel

In the Mcl plugin the Mcl Flexio Common must be checked.

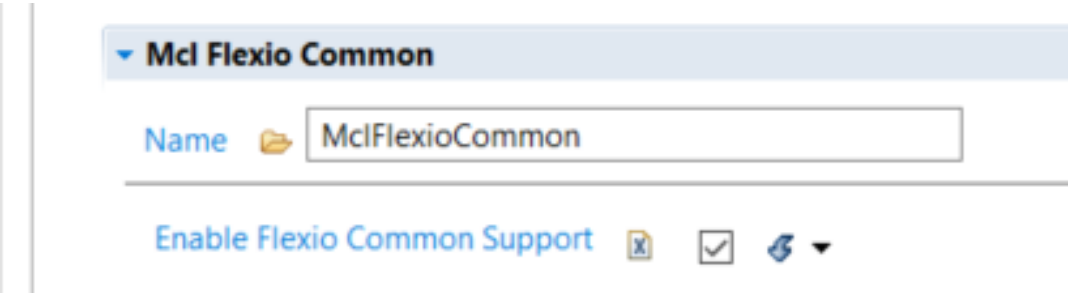

Figure 3.11 Mcl Flexio Common check.

After enabling the Mcl Flexio Common, go to Mcl/MclConfig/FlexioCommon/FlexioCommon_0/FlexioMclLogic↩ Channels and configure two Flexio Logic Channels. Make sure the selection of pins and channels is not the same between the two logic channels. One good helpful piece of advice is to use the Name field to keep easily track which channel is configured for Reception (Rx) and which one is used for Transmission (Tx). The pin chosen is the pin of the FLEXIO module which must be also added in the Port / Siul configuration.

Figure 3.12 Mcl Flexio Logic channels.

In the Lin component you must choose the Lin hardware channel one of the FLEXIO channels. The next step is to enable the Lin Flexio Rx Channel and Lin Flexio Tx Channel and selectthe Mcl Flexio Logic Channels configurted in the Mcl component. The same channel must **NOT** be configured for both Tx and Rx.

Note

Not enabling the Flexio Rx and Tx channel when using FLEXIO hardware unit is leading to build errors.

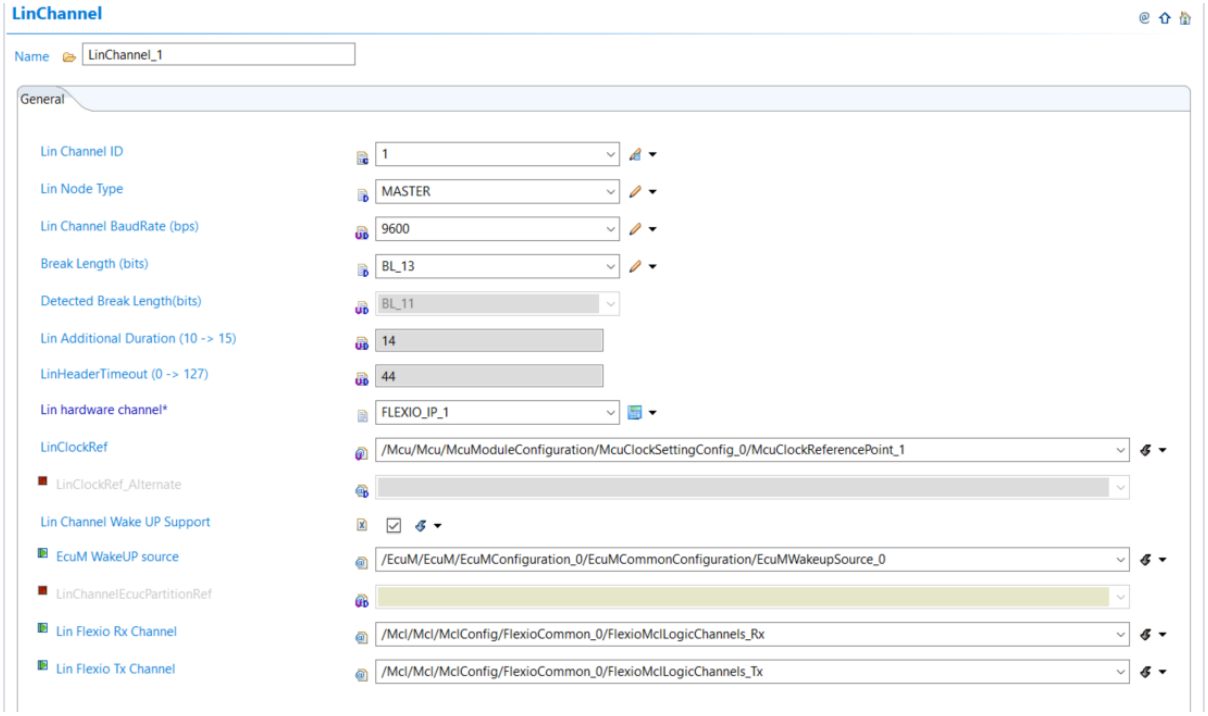

Figure 3.13 Configuration for LIN channel using FLEXIO hardware unit.

Note

All the above tips apply also for Design Studio configuration.

3.6.6 How to configure MultiPartition

Set MultiPartition support node of Lin module is ON to partitions can be referred.

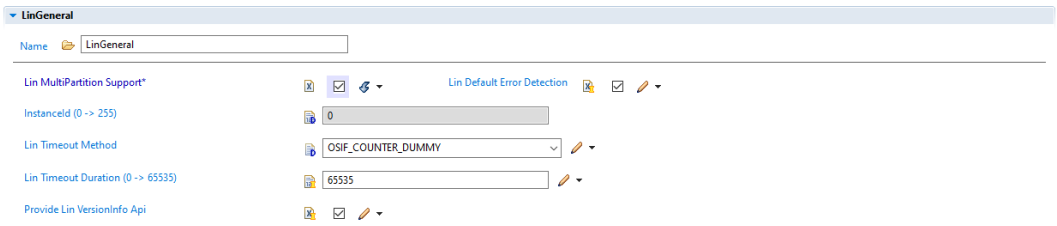

Figure 3.14 Enable MultiPartition support for Lin module.

A partition will be assigned for each Lin channel. Each LinChannel selects only a partition (reference from ECUC).

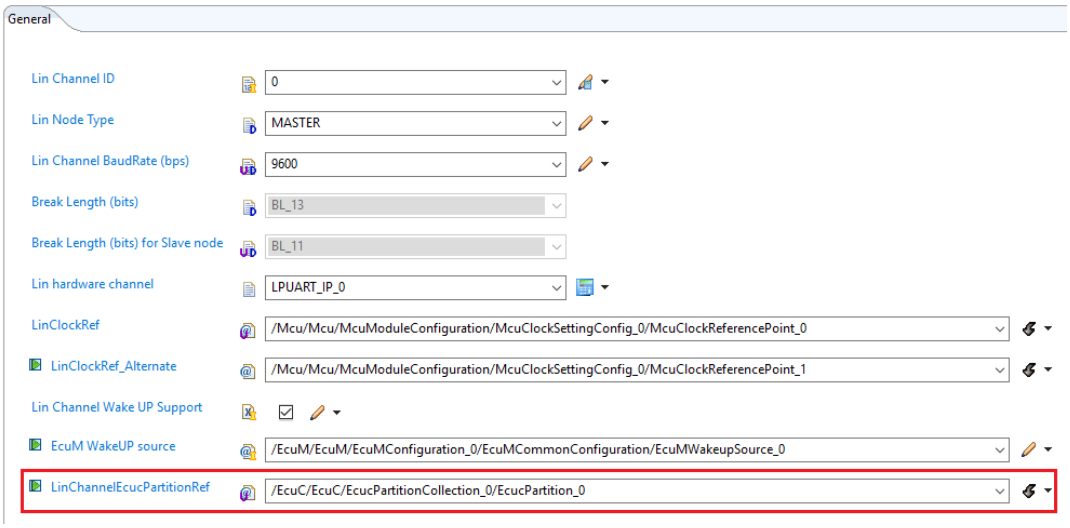

Figure 3.15 Select partition for Lin channel.

In Os configuration, Each OsApplication will be set to one partition.

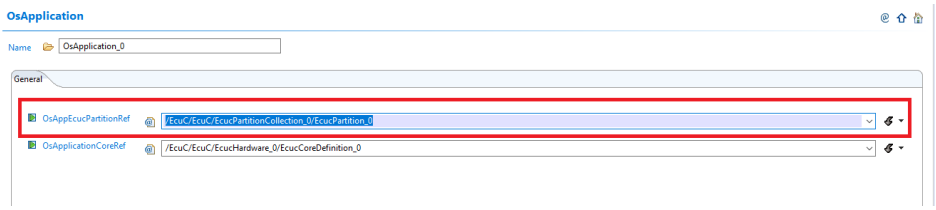

Figure 3.16 Partition referred in Os.

In EcuC configuration, the EcucPartitionCollection displays a collection of partitions that can be selected for the Lin channel.

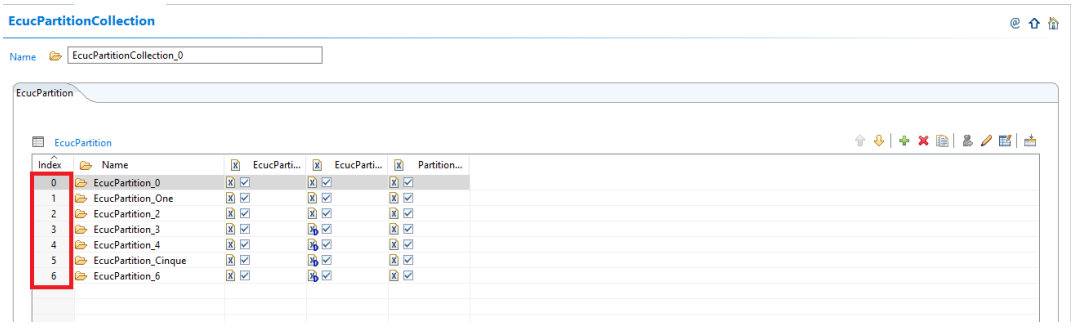

Figure 3.17 List of Partitions on EcuC.

3.7 Runtime errors

The Lin driver generates the following DEM errors at runtime.

Function	Error Code	Condition triggering the error
Lin_43_LPUART_FLEXIO_Init()	Lin_E_TIMEOUT	Timeout caused by hardware error waiting for entering Init mode and Sleep mode.

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like: *#define <Mip>Conf_<Container_ShortName>_<Container_ID>*

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

3.9 Usage in non AUTOSAR context

For the non-ASR environments, both High-Level and IP layer interfaces are exposed and can be used by software units above. These may consist of either higher level NXP SW deliverables, like stacks and middleware, or the application code itself.

The goal of the product is to be versatile, easy to use and integrate with other SW environments, offering both standardized and optimized low-level interfaces.

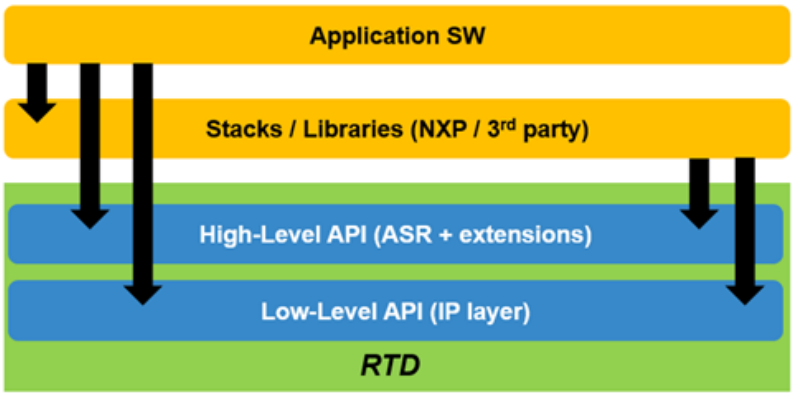

Figure 3.18 RTD Usage in non-ASR Context

In the non-AUTOSAR environment, there are no specific ECU states defined, but an external assumption for the application is that the correct sequence of initialization takes place prior to calling any runtime APIs.

The constraints considered for defining the APIs in the non-AUTOSAR environment are:

- Hardware resources used by the drivers must be properly initialized prior to runtime usage by calling the initialization function with configuration parameters generated by the configuration tools
- All dependencies must be initialized prior to the usage of a driver that depends on them; this translates to a general initialization sequence applicable for all applications, as follows:
  1. Initialization of the device clock tree so other periphery can be initialized
  2. Configuration of the pinout so peripherals with external connections can communicate with the outside world
  3. Configuration of the common resources (e.g. DMA channels, memory regions, access rights, etc) prior to the usage of any runtime functionality that has to benefit from this overall SoC configuration
  4. Initialization of specific drivers

5. Runtime functionality
6. Deinitialization (inverse initialization order)

High-level and IP layer functionalities cannot be used simultaneously over the same hardware resource. The granularity of the resource must be defined specifically for each driver, but as a general rule the default granularity considered is the peripheral instance (e.g. Gpt high level APIs cannot be called interchangeably with Pit_Ip low level APIs over the same PIT hardware instance).

The diagram below shows a generic, high-level sequence of calls to RTD interfaces in the non-AUTOSAR scenario:

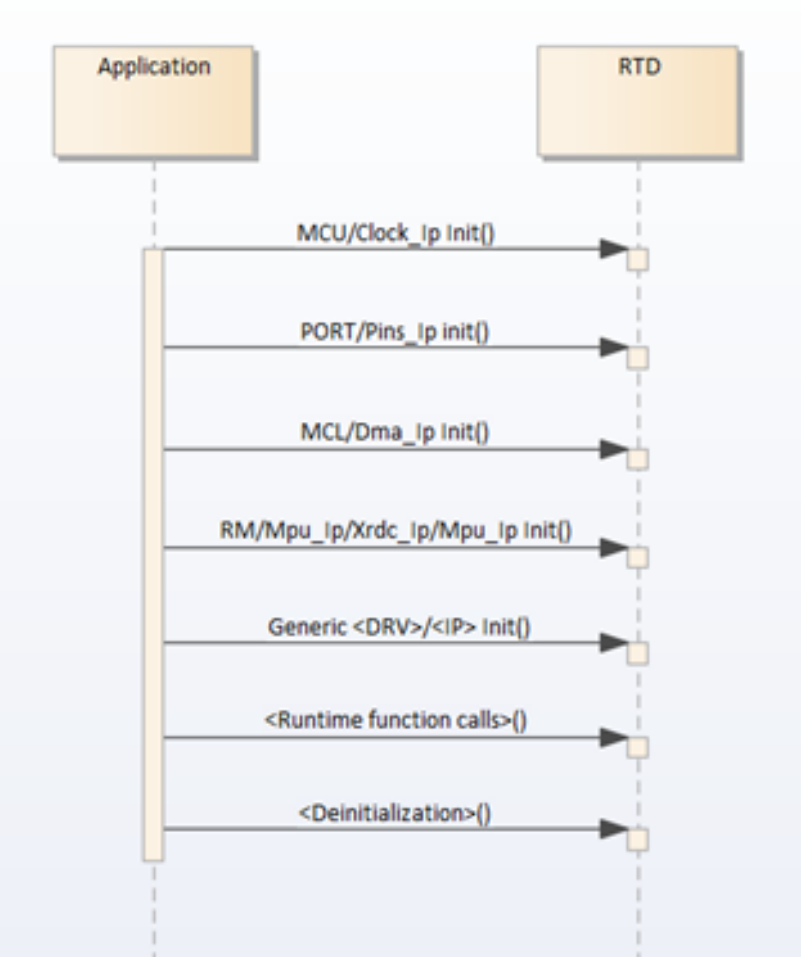

Figure 3.19 Non-AUTOSAR Sequence diagram

Chapter 4

Configuration Parameters

RTD provides configuration schema for the high level driver in EB Tresos proprietary format (**.xdm**), S32CT proprietary format (**.component**), but also in a standardized AUTOSAR format (**.epd**). For the low level driver configuration only the S32CT proprietary format (**.component**) is supported. The configuration data of the project can be saved in EB Tresos proprietary format (**xdm**) and AUTOSAR standardized format (**.epc**) from EB Tresos and in S32CT proprietary format (**.mex**) and AUTOSAR standardized format (**.epc**) from S32 Design Studio. RTD provides templates for parsing the configuration data and generating configuration structures from it in XPath format, to be used with the EB Tresos generator and in JS format, to be used with the S32CT generator. Since the generation process is not standardized by AUTOSAR, only the two mentioned tools can be used with the provided templates. Using the same input configuration data, the same configuration structures will be obtained using any of the generators.

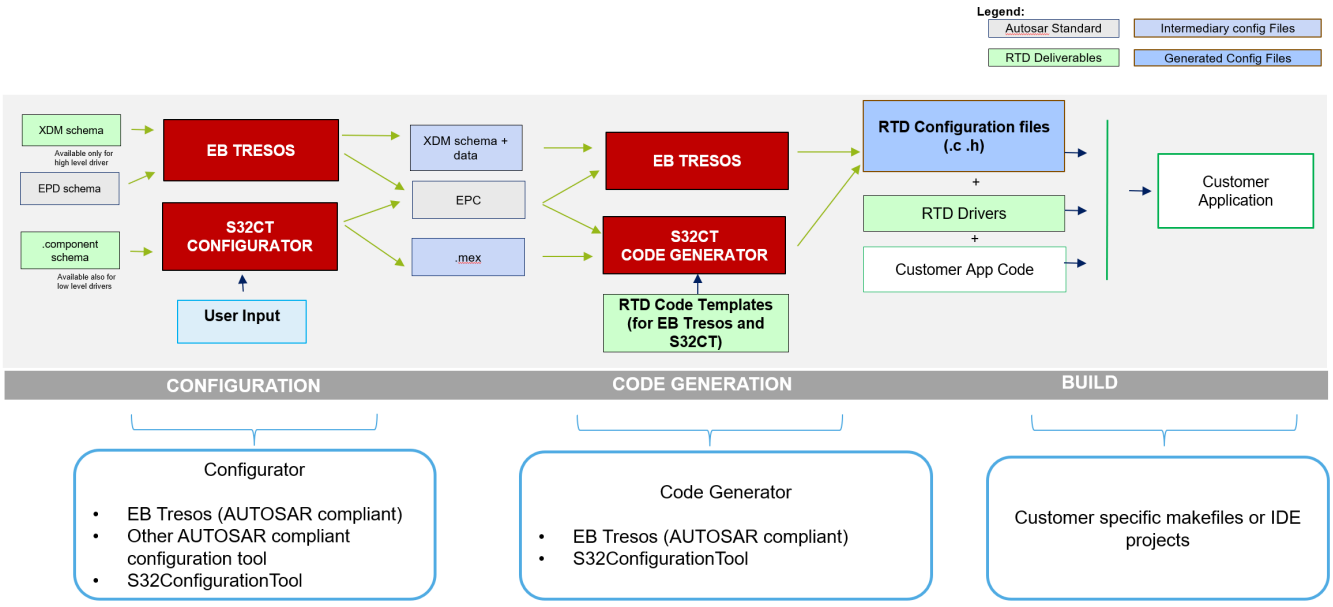

Figure 4.1 Configuration generation flow

- Lin
- LPUART_LIN
- FLEXIO_LIN

4.1 Lin

- Module Lin
  - Container AutosarExt
    - Parameter LinDisableDemReportErrorStatus
    - Parameter LinFrameTimeoutDisable
    - Parameter LinEnableUserModeSupport
    - Parameter LinLpuartStartTimerNotification
    - Parameter LinLpuartStopTimerNotification
    - Parameter LinFlexioStartTimerNotification
    - Parameter LinFlexioStopTimerNotification
    - Parameter LinLpuartWakeupTimerNotification
  - Container LinGeneral
    - Parameter LinMultiPartitionSupport
    - Parameter LinDevErrorDetect
    - Parameter LinIndex
    - Parameter LinTimeoutMethod
    - Parameter LinTimeoutDuration
    - Parameter LinVersionInfoApi
    - Reference LinEcucPartitionRef
  - Container LinDemEventParameterRefs
    - Reference LIN_E_TIMEOUT
  - Container LinGlobalConfig
    - Container LinChannel
      - Parameter LinChannelId
      - Parameter LinNodeType
      - Parameter LinChannelBaudRate
      - Parameter BreakLength
      - Parameter DetectedBreakLength
      - Parameter LinResponseTimeout
      - Parameter LinHeaderTimeout
      - Parameter LinHwChannel
      - Parameter LinChannelWakeupSupport
      - Reference LinClockRef
      - Reference LinClockRef_Alternate
      - Reference LinChannelEcuMWakeupSource
      - Reference LinChannelEcucPartitionRef

- Reference LinFlexioRxControllerRef
- Reference LinFlexioTxControllerRef

– Container CommonPublishedInformation

* Parameter ArReleaseMajorVersion
* Parameter ArReleaseMinorVersion
* Parameter ArReleaseRevisionVersion
* Parameter ModuleId
* Parameter SwMajorVersion
* Parameter SwMinorVersion
* Parameter SwPatchVersion
* Parameter VendorApiInfix
* Parameter VendorId

4.1.1 Module Lin

Configuration of the Lin (Local Interconnect Network) module.

Included containers:

- AutosarExt
- LinGeneral
- LinDemEventParameterRefs
- LinGlobalConfig
- CommonPublishedInformation

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD, VARIANT-PRE-COMPILE

4.1.2 Container AutosarExt

AutosarExt

Autosar Requirements:

This container contains the global configuration parameters of the Non-Autosar Lin driver.

This container is a MultipleConfigurationContainer, i.e. this container and

its sub-containers exist once per configuration set.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.3 Parameter LinDisableDemReportErrorStatus

LinDisableDemReportErrorStatus

Switches the Diagnostic Error Reporting and Notification OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.4 Parameter LinFrameTimeoutDisable

LinFrameTimeoutDisable

The master will accept the frame that is longer than TFrame_Maximum.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.5 Parameter LinEnableUserModeSupport

When this parameter is enabled, the MDL module will adapt to run from User Mode, with the following measures:

a) configuring REG_PROT for ABC1, ABC2 IPs so that the registers under protection can be accessed from user mode by setting UAA bit in REG_PROT_GCR to 1

b) using 'call trusted function' stubs for all internal function calls that access registers requiring supervisor mode.

c) other module specific measures

for more information, please see chapter 5.7 User Mode Support in IM

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.6 Parameter LinLpuartStartTimerNotification

Lin Start Timer Notification for the checking frame timeout error when Lin over Lpuart used.

This parameter is a reference to a notification function to start timer when reception has just begun.

Note: This parameter should be configured when LinFrameTimeoutDisable is OFF

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.1.7 Parameter LinLpuartStopTimerNotification

Lin Stop Timer Notification for the checking frame timeout error when Lin over Lpuart used.

This parameter is a reference to a notification function to stop timer.

Note: This parameter should be configured when LinFrameTimeoutDisable is OFF

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.1.8 Parameter LinFlexioStartTimerNotification

Lin Start Timer Notification for the checking frame timeout error when Lin over Flexio used.

This parameter is a reference to a notification function to start timer when reception has just begun.

Note: This parameter should be configured when LinFrameTimeoutDisable is OFF

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.1.9 Parameter LinFlexioStopTimerNotification

Lin Stop Timer Notification for the checking frame timeout error when Lin over Flexio used.

This parameter is a reference to a notification function to stop timer when reception finished or timeout occurred.

Note: This parameter should be configured when LinFrameTimeoutDisable is OFF

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.1.10 Parameter LinLpuartWakeupTimerNotification

Lin Lpuart Wake UP Timer Notification for the checking Wake Up pulse when Lin over Lpuart used.

This parameter is a reference to a notification function to start/stop timer corresponding falling/rising of Wakeup pulse.

Note: This parameter have to be configured when sending/receiving the wakeup pulse.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.1.11 Container LinGeneral

LinGeneral

Autosar Requirements: ECUC_Lin_00183

This container contains the parameters related to each LIN Driver Unit.

This container is a MultipleConfigurationContainer, i.e. this container and

its sub-containers exit once per configuration set.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.12 Parameter LinMultiPartitionSupport

This parameter determine multi-core feature will be used in Lin driver.

If LinMultiPartitionSupport is disabled, then for all the variants no partition shall be defined.

If LinMultiPartitionSupport is enabled, at least one EcucPartition needs to be defined (in all variants).

Note: S32K1XX/S32M24X does not support multi-core feature.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.13 Parameter LinDevErrorDetect

LinDevErrorDetect

Autosar Requirements: ECUC_Lin_00066

Switches the Default Error Detection and Notification ON or OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

Property	Value
defaultValue	false

4.1.14 Parameter LinIndex

Autosar Requirements: ECUC_Lin_00179

Specifies the InstanceId of this module instance. If only one instance is present it shall have the Id 0.

Note, this parameter is not used in the current implementation.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	255
min	0

4.1.15 Parameter LinTimeoutMethod

LinTimeoutMethod

Configures the timeout method.

Based on this selection a certain timeout method from OsIf will be used in the driver.

This parameter is used to select between different OsIf counter implementations. For additional details, please refer to the OsIf documentation.

Note: If SystemTimer or CustomTimer are selected make sure the corresponding timer is enabled in OsIf General configuration.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	OSIF_COUNTER_DUMMY
literals	['OSIF_COUNTER_DUMMY', 'OSIF_COUNTER_SYSTEM', 'OSIF_COU↩ NTER_CUSTOM']

4.1.16 Parameter LinTimeoutDuration

Specifies the maximum number of loops for blocking function until a timeout is raised in short term wait loops. If LinTimeoutMethod is OSIF_COUNTER_SYSTEM or OSIF_COUNTER_CUSTOM, LinTimeoutDuration is in microsecond value. If LinTimeoutMethod is OSIF_COUNTER_DUMMY, the LinTimeoutDuration is number of wait loop.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1000
max	65535
min	0

4.1.17 Parameter LinVersionInfoApi

LinVersionInfoApi

Autosar Requirements: ECUC_Lin_00067

Switches the Lin_GetVersionInfo function ON or OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.18 Reference LinEcucPartitionRef

Maps the Lin driver to zero or multiple ECUC partitions to make the modules API available in this partition.

The Lin driver will operate as an independent instance in each of the partitions.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.1.19 Container LinDemEventParameterRefs

Container for the references to DemEventParameter elements which shall be invoked using the API Dem_SetEventStatus API in case the corresponding error occurs. The EventId is taken from the referenced DemEventParameter's DemEventId value. The standardized errors are provided in the container and can be extended by vendor specific error references.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.20 Reference LIN_E_TIMEOUT

Reference to the DemEventParameter which shall be issued when the error "Timeout caused by hardware error" has occured.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/Dem/DemConfigSet/DemEventParameter

4.1.21 Container LinGlobalConfig

This container contains the global configuration parameter of the Lin driver. This container is a MultipleConfigurationContainer, i.e. this container and its sub-containers exit once per configuration set.

Included subcontainers:

- LinChannel

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.22 Container LinChannel

This container contains the configuration (parameters) of the LIN Controller(s).

Note:"User should use unique names for naming the LIN channels across different LinGlobalConfig Sets."

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.23 Parameter LinChannelId

Identifies the LIN channel. Replaces LIN_CHANNEL_INDEX_NAME from the LIN SWS.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC

Property	Value
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	65535
min	0

4.1.24 Parameter LinNodeType

LinNodeType

Autosar Requirements: ECUC_Lin_00191

Specifies the LIN node type of this channel: Master or Slave node

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	MASTER
literals	['MASTER', 'SLAVE']

4.1.25 Parameter LinChannelBaudRate

LinChannelBaudRate

Autosar Requirements: ECUC_Lin_00180

Specifies the baud rate of the LIN channel in 'bps'. Valid range: 1000..20000.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	9600
max	20000
min	1000

4.1.26 Parameter BreakLength

Defines the break length in bits.

Note:

This Parameter is an Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	BL_13
literals	['BL_13']

4.1.27 Parameter DetectedBreakLength

Defines the break length in bits which can detect by Lin node in Slave mode.

Note:

This Parameter is an Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	BL_11
literals	['BL_11']

4.1.28 Parameter LinResponseTimeout

Response timeout value.

This is the response timeout duration (in bit time) for 1 byte.

The default value is 14, corresponding to T_Response_Maximum = 1.4 x T_Response_Nominal. Here, 1.4 corresponds to LinResponseTimeout/10.

T_Response_Nominal = 10 x (N + 1) x T, where N is number of data, T is the nominal time required to transmit a bit.

This node is only configured if AutosarExt/LinFrameTimeoutDisable is false.

Note:

This Parameter is an Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	14
max	20
min	0

4.1.29 Parameter LinHeaderTimeout

Header timeout value.

This field contains the header timeout duration (in bit time) after slave detect break character.

Example:

- Header_nominal = 13 + 2 + 10 + 10 = 35 (Bit time)

*- Additional 40% duration compared to the nominal transmission time, so Header_max_time = 1.4 * Header_nominal = 49 (Bit time)*

*- Taking into account a possible 14% clock deviation, header_max seen is 49 * 1.14 = 56 (Bit time)*

- The counter start after detection break - 11 Bit time, the header timeout value is 56 - 11 = 45

This node is only configured if AutosarExt/LinFrameTimeoutDisable is false and LinNodeType is SLAVE.

Note:

This Parameter is an Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	44
max	127
min	0

4.1.30 Parameter LinHwChannel

Selects the Lin Hardware Channel.

Note:

This Parameter is an Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	LPUART_IP_0
literals	['LPUART_IP_0', 'LPUART_IP_1', 'FLEXIO_IP_0', 'FLEXIO_IP_1']

4.1.31 Parameter LinChannelWakeupSupport

LinChannelWakeupSupport

Autosar Requirements: LIN182_Conf

Specifies if the LIN hardware channel supports wake up functionality.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.32 Reference LinClockRef

Reference to the LIN clock source configuration, which is set in the MCU driver configuration.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/Mcu/McuModuleConfiguration/McuClockSetting↩ Config/McuClockReferencePoint

4.1.33 Reference LinClockRef_Alternate

Alternate reference to the LIN clock source configuration, which is set in the MCU driver configuration, used in Low Power Mode.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/Mcu/McuModuleConfiguration/McuClockSetting↩ Config/McuClockReferencePoint

4.1.34 Reference LinChannelEcuMWakeupSource

This parameter contains a reference to the Wakeup Source for this controller as defined in the ECU State Manager.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuM/EcuMConfiguration/EcuMCommon↩ Configuration/EcuMWakeupSource

4.1.35 Reference LinChannelEcucPartitionRef

Maps one single Lin channel to zero or one ECUC partitions.

The ECUC partition referenced is a subset of the ECUC partitions where the Lin driver is mapped to.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.1.36 Reference LinFlexioRxControllerRef

Lin Flexio Rx Channel Reference

Reference to the Flexio Controller configure for the Reception

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destinations	['/TS_T40D2M30I0R0/Mcl/MclConfig/FlexioCommon/FlexioMclLogicChannels']

4.1.37 Reference LinFlexioTxControllerRef

Lin Flexio Tx Channel Reference

Reference to the Flexio Controller configure for the Transmission

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destinations	['/TS_T40D2M30I0R0/Mcl/MclConfig/FlexioCommon/FlexioMclLogicChannels']

4.1.38 Container CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.39 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.1.40 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	7
max	7
min	7

4.1.41 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.42 Parameter ModuleId

Module ID of this module from Module List.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	82
max	82
min	82

4.1.43 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	3
max	3
min	3

4.1.44 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.45 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION

Property	Value
defaultValue	0
max	0
min	0

4.1.46 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

<ModuleName>_>VendorId>_<VendorApiInfix>.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	LPUART_FLEXIO

4.1.47 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43

Property	Value
max	43
min	43

4.2 LPUART_LIN

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module Lpuart_Lin
  - Container LinGeneral
    - Parameter LinDevErrorDetect
    - Parameter LinIndex
    - Parameter LinTimeoutDuration
    - Parameter LinTimeoutMethod
    - Parameter LinFrameTimeoutDisable
    - Parameter LinLpuartStartTimerNotification
    - Parameter LinLpuartStopTimerNotification
    - Parameter LinLpuartWakeupTimerNotification
    - Parameter LinLpuartEnableAutobaudFeature
    - Parameter LinLpuartSignalMeasurementCallback
    - Parameter LinFlexioStartTimerNotification
    - Parameter LinFlexioStopTimerNotification
  - Container LinGlobalConfig
    - Container LinChannel
      - Parameter LinChannelId
      - Parameter LinNodeType
      - Parameter LinChannelBaudRate
      - Parameter BreakLength
      - Parameter DetectedBreakLength
      - Parameter LinResponseTimeout
      - Parameter LinHeaderTimeout
      - Parameter LinHwChannel
      - Parameter LinUserCallback
      - Parameter LinLpuartAutobaudEnable
      - Parameter LinClockRef

4.2.1 Module Lpuart_Lin

Configuration of the Lin (Lin Driver) module.

Included containers:

- LinGeneral
- LinGlobalConfig

Property	Value
type	MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD;VARIANT-PRE-COMPILE

4.2.2 Parameter LinLpuartEnableAutobaudFeature

* Enable Autobaud feature *

* This feature should be only used for slave mode and use to calculate and update new baudrate when the slave node is received header frame from Master node. *

Property	Value
type	BOOLEAN
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.2.3 Parameter LinLpuartSignalMeasurementCallback

Callback function to start/stop input capture mode of the FTM timer which counts the duration between a rising and falling edge in order to measure the baudrate based on the sync byte in nanoseconds. This Callback should be only used for slave mode and LinLpuartEnableAutobaudFeature node of channel is enabled.

Property	Value
type	STRING

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	NULL_PTR

4.2.4 Parameter LinUserCallback

Callback function to invoke after receiving a byte or transmitting a byte.

Property	Value
type	STRING
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	NULL_PTR

4.2.5 Parameter LinLpuartAutobaudEnable

* Enable Autobaud feature *

* This feature should be only used for slave and use to calculate and update new baudrate when the slave mode is received header frame. This is node will only enable when LinLpuartEnableAutobaudFeature is enabled. *

* Node: If the auto baudrate is enabled for the channel, the channel baudrate will be set to 19200 to be able to*

*do not miss any falling edge in sync byte. *

Property	Value
type	BOOLEAN
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.3 FLEXIO_LIN

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module Flexio_Lin
  - Container LinGeneral
    - Parameter LinDevErrorDetect
    - Parameter LinIndex
    - Parameter LinTimeoutDuration
    - Parameter LinTimeoutMethod
    - Parameter LinFrameTimeoutDisable
    - Parameter LinLpuartStartTimerNotification
    - Parameter LinLpuartStopTimerNotification
    - Parameter LinLpuartWakeupTimerNotification
    - Parameter LinFlexioStartTimerNotification
    - Parameter LinFlexioStopTimerNotification
  - Container LinGlobalConfig
    - Container LinChannel
      - Parameter LinChannelId
      - Parameter LinNodeType
      - Parameter LinChannelBaudRate
      - Parameter BreakLength
      - Parameter DetectedBreakLength
      - Parameter LinResponseTimeout
      - Parameter LinHeaderTimeout
      - Parameter LinHwChannel
      - Parameter LinUserCallback
      - Parameter LinClockRef
      - Reference LinFlexioRxControllerRef

- Reference LinFlexioTxControllerRef

4.3.1 Module Flexio_Lin

Configuration of the Lin (Lin Driver) module.

Included containers:

- LinGeneral
- LinGlobalConfig

Property	Value
type	MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD;VARIANT-PRE-COMPILE

4.3.2 Parameter LinUserCallback

Callback function to invoke after receiving a byte or transmitting a byte.

Property	Value
type	STRING
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	NULL_PTR

Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

Chapter 6

Module Documentation

6.1 FLEXIO_IP

6.1.1 Detailed Description

Data Structures

- struct Flexio_Lin_Ip_PduType

  This Type is used to provide PID, checksum model, response type of the frame, data length and SDU pointer from the LIN Interface to the Flexio Lin Ip driver. More...

- struct Flexio_Lin_Ip_StateStructType

  Flexio Lin Ip driver state structure type. It is internal for the driver use only. Implements : Flexio_Lin_Ip_State↩ StructType_Class. More...

- struct Flexio_Lin_Ip_UserConfigType

  Flexio Lin Ip driver User configuration structure type @Implements : Flexio_Lin_Ip_UserConfigType_Class. More...

Macros

- #define FLEXIO_LIN_IP_SLAVE

  Macro that specifies usage of a SLAVE note.

- #define FLEXIO_LIN_IP_MASTER

  Macro that specifies usage of a SLAVE note.

Types Reference

- typedef void(∗ Flexio_Lin_Ip_CallbackType) (const uint8 Instance, const Flexio_Lin_Ip_StateStructType ∗LinState)

  LIN Driver callback function type.

Enum Reference

- enum Flexio_Lin_Ip_EventIdType

  Defines types for an enumerating event related to an Identifier.

- enum Flexio_Lin_Ip_NodeStateType

  Define type for an enumerating LIN Node state.

- enum Flexio_Lin_Ip_StatusType

Define type function possible return values.

- enum Flexio_Lin_Ip_FrameCsModelType

  Define type for check sum type of the frame.

- enum Flexio_Lin_Ip_FrameResponseType

  Define type of the response part of frame.

- enum Flexio_Lin_Ip_TransferStatusType

  Description: This type defines a range of transfer status.

Function Reference

- Flexio_Lin_Ip_StatusType Flexio_Lin_Ip_Init (const uint8 Channel, const Flexio_Lin_Ip_UserConfigType *UserConfig)

  Initializes an LIN_FLEXIO channel for LIN Network.

- void Flexio_Lin_Ip_GoToIdleState (const uint8 Channel)

  Puts current LIN node to Idle state This function changes current node state to FLEXIO_LIN_IP_NODE_ST↩ ATE_IDLE.

- Flexio_Lin_Ip_StatusType Flexio_Lin_Ip_AbortTransferData (const uint8 Channel)

  Aborts an on-going non-blocking transmission/reception.

- Flexio_Lin_Ip_StatusType Flexio_Lin_Ip_GoToSleepMode (const uint8 Channel)

  This function puts current node to sleep mode.

- Flexio_Lin_Ip_StatusType Flexio_Lin_Ip_SendWakeupSignal (const uint8 Channel)

  Sends a wakeup signal through the LIN_FLEXIO interface.

- Flexio_Lin_Ip_StatusType Flexio_Lin_Ip_Deinit (const uint8 Channel)

  De-initialize a FLEXIO LIN channel.

- Flexio_Lin_Ip_TransferStatusType Flexio_Lin_Ip_GetStatus (const uint8 Channel, const uint8 **Buffer)

  Returns the status of an on going transmission.

- Flexio_Lin_Ip_StatusType Flexio_Lin_Ip_SendFrame (const uint8 Channel, const Flexio_Lin_Ip_PduType *PduInfo)

  Sends a LIN frame as master or a response as a slave.

- void Flexio_Lin_Ip_TimerExpiredService (const uint8 Channel)

  This is callback function for Timer Interrupt Handler. Users shall initialize a timer (for example FTM) and the time period in microseconds will be set by the driver via LinFlexioStartTimerNotification. In timer IRQ handler, call this function.

6.1.2 Data Structure Documentation

6.1.2.1 struct Flexio_Lin_Ip_PduType

This Type is used to provide PID, checksum model, response type of the frame, data length and SDU pointer from the LIN Interface to the Flexio Lin Ip driver.

Definition at line 207 of file Flexio_Lin_Ip_Types.h.

Data Fields

- uint8 Pid

  LIN frame identifier.
- Flexio_Lin_Ip_FrameCsModelType Cs

  Checksum model type.
- Flexio_Lin_Ip_FrameResponseType Drc

  Response type.
- uint8 Dl

  Data length.
- uint8 * SduPtr

  Pointer to Sdu.

Field Documentation

Pid

`uint8 Pid`

LIN frame identifier.

Definition at line 209 of file Flexio_Lin_Ip_Types.h.

Cs

`Flexio_Lin_Ip_FrameCsModelType Cs`

Checksum model type.

Definition at line 210 of file Flexio_Lin_Ip_Types.h.

Drc

`Flexio_Lin_Ip_FrameResponseType Drc`

Response type.

Definition at line 211 of file Flexio_Lin_Ip_Types.h.

Dl

`uint8 Dl`

Data length.

Definition at line 212 of file Flexio_Lin_Ip_Types.h.

SduPtr

```
uint8* SduPtr
```

Pointer to Sdu.

Definition at line 213 of file Flexio_Lin_Ip_Types.h.

6.1.2.2 struct Flexio_Lin_Ip_StateStructType

Flexio Lin Ip driver state structure type. It is internal for the driver use only. Implements : Flexio_Lin_Ip_State↩ StructType_Class.

Definition at line 221 of file Flexio_Lin_Ip_Types.h.

Data Fields

uint8	CntByte	To count number of bytes already transmitted or received.
uint8	CheckSum	CheckSum byte.
volatile boolean	IsBusBusy	True if there are data, frame headers being transferred on bus.
uint8 *	SduBuf	The buffer of received data.
volatile uint8	Dl	The remaining number of bytes to be transmitted.
uint8	CurrentId	Current ID.
uint8	CurrentPid	Current PID.
Flexio_Lin_Ip_FrameResponseType	FrameCommand	
Flexio_Lin_Ip_FrameCsModelType	Cs	Current Checksum type.
volatile Flexio_Lin_Ip_EventIdType	CurrentEventId	Current ID Event.
volatile Flexio_Lin_Ip_NodeStateType	CurrentNodeState	Current Node state.
volatile Flexio_Lin_Ip_NodeStateType	PreviousNodeState	Store previous node state when set Lin channel to idle for further processing.

6.1.2.3 struct Flexio_Lin_Ip_UserConfigType

Flexio Lin Ip driver User configuration structure type @Implements : Flexio_Lin_Ip_UserConfigType_Class.

Definition at line 252 of file Flexio_Lin_Ip_Types.h.

Data Fields

uint8	Instance	Flexio hardware channel for this configuration.
uint8	TxShifterId	Flexio shifter use for transmission(Tx).
uint8	TxTimerId	Flexio timer use for transmission(Tx).
uint8	TxPin	Flexio pin use for transmission(Tx).
uint8	RxShifterId	Flexio shifter use for reception(Rx).
uint8	RxTimerId	Flexio timer use for reception(Rx).
uint8	RxPin	Flexio pin use for reception(Rx).

Data Fields

uint8	FlexioHwInstance	Flexio hardware module number.
uint8	MasterBreakLength	Number of bits for break length.
uint8	SlaveSyncBreakLength	Number of bits detected by slave node(break length).
uint8	TimerClkSrc	Timer clock source.
uint16	Baudratedivider	Baudrate divider.
boolean	NodeFunction	Type of Lin node. It can be Master node or Slave node.
Flexio_Lin_Ip_CallbackType	Callback	Callback function to invoke after receiving a byte or transmitting a byte.
Flexio_Lin_Ip_StateStructType *	StateStruct	Pointer for the internal state structure of the driver.
uint32	Baudrate	Baudrate value configured.
uint8	WakeupByte	Byte will be sent to generate wakeup pulse [450us->5ms].
uint8	BaseWakeupByteDetectInverted	Byte use to check detection of wake up pulse longer than 150us.

6.1.3 Macro Definition Documentation

6.1.3.1 FLEXIO_LIN_IP_SLAVE

```
#define FLEXIO_LIN_IP_SLAVE
```

Macro that specifies usage of a SLAVE note.

This macro is used in user configuration structure to set the node type as SLAVE.

Definition at line 92 of file Flexio_Lin_Ip_Types.h.

6.1.3.2 FLEXIO_LIN_IP_MASTER

```
#define FLEXIO_LIN_IP_MASTER
```

Macro that specifies usage of a SLAVE note.

This macro is used in user configuration structure to set the node type as MASTER.

Definition at line 100 of file Flexio_Lin_Ip_Types.h.

6.1.4 Types Reference

6.1.4.1 Flexio_Lin_Ip_CallbackType

```
typedef void(* Flexio_Lin_Ip_CallbackType) (const uint8 Instance, const Flexio_Lin_Ip_StateStructType
*LinState)
```

LIN Driver callback function type.

Definition at line 246 of file Flexio_Lin_Ip_Types.h.

6.1.5 Enum Reference

6.1.5.1 Flexio_Lin_Ip_EventIdType

enum Flexio_Lin_Ip_EventIdType

Defines types for an enumerating event related to an Identifier.

Enumerator

FLEXIO_LIN_IP_NO_EVENT	No event.
FLEXIO_LIN_IP_WAKEUP_SIGNAL	Wakeup signal detected.
FLEXIO_LIN_IP_BAUDRATE_ADJUSTED	Baudrate adjusted.
FLEXIO_LIN_IP_RECV_BREAK_FIELD_OK	Break field received.
FLEXIO_LIN_IP_SYNC_ERROR	Sync byte received with errors.
FLEXIO_LIN_IP_RECV_HEADER_OK	PID byte received ok.
FLEXIO_LIN_IP_PID_ERROR	PID byte received with errors.
FLEXIO_LIN_IP_TX_UNDERRUN_ERROR	Tx underrun error.
FLEXIO_LIN_IP_READBACK_ERROR	Readback error.
FLEXIO_LIN_IP_CHECKSUM_ERROR_EVENT	CheckSum error.
FLEXIO_LIN_IP_TX_COMPLETED	Tx completed.
FLEXIO_LIN_IP_RX_COMPLETED	Rx completed.
FLEXIO_LIN_IP_RX_OVERRUN_ERROR	Rx overrun error.
FLEXIO_LIN_IP_TIMEOUT_ERROR	Timeout error.

Definition at line 107 of file Flexio_Lin_Ip_Types.h.

6.1.5.2 Flexio_Lin_Ip_NodeStateType

enum Flexio_Lin_Ip_NodeStateType

Define type for an enumerating LIN Node state.

Enumerator

FLEXIO_LIN_IP_NODE_STATE_UNINIT	Uninitialized state.
FLEXIO_LIN_IP_NODE_STATE_SLEEP_MODE	Sleep mode state.
FLEXIO_LIN_IP_NODE_STATE_IDLE	Idle state.
FLEXIO_LIN_IP_NODE_STATE_SEND_BREAK_FIELD	Send break field state.
FLEXIO_LIN_IP_NODE_STATE_RECV_SYNC	Receive the synchronization byte state.
FLEXIO_LIN_IP_NODE_STATE_SEND_PID	Send PID state.
FLEXIO_LIN_IP_NODE_STATE_RECV_PID	Receive PID state.
FLEXIO_LIN_IP_NODE_STATE_RECV_DATA	Receive data state.
FLEXIO_LIN_IP_NODE_STATE_RECV_DATA_COMPLETED	Receive data completed state.

Enumerator

FLEXIO_LIN_IP_NODE_STATE_SEND_DATA	Send data state.
FLEXIO_LIN_IP_NODE_STATE_SEND_DATA_COMPLETED	Send data completed state.
FLEXIO_LIN_IP_NODE_STATE_SEND_SYNC	Send data completed state.

Definition at line 129 of file Flexio_Lin_Ip_Types.h.

6.1.5.3 Flexio_Lin_Ip_StatusType

enum Flexio_Lin_Ip_StatusType

Define type function possible return values.

Definition at line 150 of file Flexio_Lin_Ip_Types.h.

6.1.5.4 Flexio_Lin_Ip_FrameCsModelType

enum Flexio_Lin_Ip_FrameCsModelType

Define type for check sum type of the frame.

Enumerator

FLEXIO_LIN_IP_ENHANCED_CS	Enhanced CheckSum model.
FLEXIO_LIN_IP_CLASSIC_CS	Classic CheckSum model.

Definition at line 161 of file Flexio_Lin_Ip_Types.h.

6.1.5.5 Flexio_Lin_Ip_FrameResponseType

enum Flexio_Lin_Ip_FrameResponseType

Define type of the response part of frame.

Enumerator

FLEXIO_LIN_IP_FRAMERESPONSE_TX	Response is generated from this node.
FLEXIO_LIN_IP_FRAMERESPONSE_RX	Response is generated from another node and is relevant for this node.
FLEXIO_LIN_IP_FRAMERESPONSE_IGNORE	Response is generated from another node and is irrelevant for this node.

Definition at line 172 of file Flexio_Lin_Ip_Types.h.

6.1.5.6 Flexio_Lin_Ip_TransferStatusType

enum Flexio_Lin_Ip_TransferStatusType

Description: This type defines a range of transfer status.

Enumerator

FLEXIO_LIN_IP_STATUS_FAIL	Command has not been accepted .
FLEXIO_LIN_IP_STATUS_TX_OK	Master: Header or Response successful transmission. Slave: Response successful transmission.
FLEXIO_LIN_IP_STATUS_TX_BUSY	Master: Ongoing transmission (Header or Response). Slave: Ongoing transmission response.
FLEXIO_LIN_IP_STATUS_TX_HEADER_ER↩ ROR	Master: Erroneous header transmission such as: Mismatch between sent and read back data or Physical bus error.
FLEXIO_LIN_IP_STATUS_TX_ERROR	Master or Slave: Erroneous response transmission such as mismatch between sent and read back data, Physical bus error.
FLEXIO_LIN_IP_STATUS_RX_OK	Master or Slave: Reception of correct response.
FLEXIO_LIN_IP_STATUS_RX_BUSY	Master or Slave: Ongoing reception. At least one response byte has been received, but the checksum byte has not been received.
FLEXIO_LIN_IP_STATUS_RX_ERROR	Master or Slave: Erroneous response reception such as: - Framing error - Overrun error - Checksum error or Short response(timeout occurred).
FLEXIO_LIN_IP_STATUS_RX_NO_RESPONSE	Master or Slave: No response byte has been received so far(timeout occurred).
FLEXIO_LIN_IP_STATUS_RX_HEADER_OK	Slave: Reception of correct header.
FLEXIO_LIN_IP_STATUS_RX_HEADER_BUSY	Slave: Ongoing header reception.
FLEXIO_LIN_IP_STATUS_RX_HEADER_ER↩ ROR	Slave: Erroneous header reception such as: Break field error, Sync field error, Identifier parity error, Framing error, timeout occurred or Physical bus error.
FLEXIO_LIN_IP_STATUS_OPERATIONAL	Normal operation; there is no data has been sent or received after channel initialized or switched to IDLE from SLEEP.
FLEXIO_LIN_IP_STATUS_SLEEP	Sleep state operation.

Definition at line 183 of file Flexio_Lin_Ip_Types.h.

6.1.6 Function Reference

6.1.6.1 Flexio_Lin_Ip_Init()

```
Flexio_Lin_Ip_StatusType Flexio_Lin_Ip_Init (
            const uint8 Channel,
            const Flexio_Lin_Ip_UserConfigType * UserConfig )
```

Initializes an LIN_FLEXIO channel for LIN Network.

The caller provides memory for the driver state structures during initialization. The user must select the LIN_FL↩ EXIO clock source in the application to initialize the LIN_FLEXIO. This function initializes a FLEXIO channel for operation. This function will initialize the run-time state structure to keep track of the on-going transfers, initialize

the module to user defined settings and default settings. It will configure two timers, two shifters and two pins for Rx and Tx transmissions and at the end will enable the FLEXIO module.

Parameters

Channel	LIN_FLEXIO channel number
UserConfig	user configuration structure of type Flexio_Lin_Ip_UserConfigType

Returns

Flexio_Lin_Ip_StatusType

Return values

FLEXIO_LIN_IP_STATUS_SUCCESS	: Init command has been accepted.
FLEXIO_LIN_IP_STATUS_ERROR	: Flexio module is not enabled, thus flexio channel cannot be initialized.

6.1.6.2 Flexio_Lin_Ip_GoToIdleState()

```
void Flexio_Lin_Ip_GoToIdleState (
            const uint8 Channel )
```

Puts current LIN node to Idle state This function changes current node state to FLEXIO_LIN_IP_NODE_ST↩ ATE_IDLE.

Parameters

channel	LIN_FLEXIO channel number

Returns

void

6.1.6.3 Flexio_Lin_Ip_AbortTransferData()

```
Flexio_Lin_Ip_StatusType Flexio_Lin_Ip_AbortTransferData (
            const uint8 Channel )
```

Aborts an on-going non-blocking transmission/reception.

While performing a non-blocking transferring data, users can call this function to terminate immediately the transferring.

Parameters

channel	LIN_FLEXIO channel number

Returns

operation status:

Return values

FLEXIO_LIN_IP_STATUS_SUCCESS	: The frame was aborted
FLEXIO_LIN_IP_STATUS_ERROR	: Operation failed due to transmission/reception could not stopped in timeout.

6.1.6.4 Flexio_Lin_Ip_GoToSleepMode()

```
Flexio_Lin_Ip_StatusType Flexio_Lin_Ip_GoToSleepMode (
            const uint8 Channel )
```

This function puts current node to sleep mode.

This function changes current node state to FLEXIO_LIN_IP_NODE_STATE_SLEEP_MODE

Parameters

channel	LIN_FLEXIO channel number

Returns

Flexio_Lin_Ip_StatusType

Return values

FLEXIO_LIN_IP_STATUS_SUCCESS	: Gotosleep command has been accepted.

6.1.6.5 Flexio_Lin_Ip_SendWakeupSignal()

```
Flexio_Lin_Ip_StatusType Flexio_Lin_Ip_SendWakeupSignal (
            const uint8 Channel )
```

Sends a wakeup signal through the LIN_FLEXIO interface.

Parameters

channel	LIN_FLEXIO channel number

Returns

operation status:

Return values

FLEXIO_LIN_IP_STATUS_SUCCESS	: Command has been accepted.
FLEXIO_LIN_IP_STATUS_ERROR	: Command has not been accepted, error occurred.

6.1.6.6 Flexio_Lin_Ip_Deinit()

```
Flexio_Lin_Ip_StatusType Flexio_Lin_Ip_Deinit (
            const uint8 Channel )
```

De-initialize a FLEXIO LIN channel.

This function shall de initialize a flexio lin channel. If no channel is available, disables also the FLEXIO module.

Parameters

channel	LIN_FLEXIO instance number.

Returns

operation status: VOID

6.1.6.7 Flexio_Lin_Ip_GetStatus()

```
Flexio_Lin_Ip_TransferStatusType Flexio_Lin_Ip_GetStatus (
            const uint8 Channel,
            const uint8 ** Buffer )
```

Returns the status of an on going transmission.

This function returns the status of the current non-blocking transfer. If a response reception has been successfully received, Buffer will be referenced to receive buffer.

Parameters

channel	LIN_FLEXIO instance number.
Buffer	Pointer to the received data buffer.

Returns

operation status: Flexio_Lin_Ip_TransferStatusType

Return values

FLEXIO_LIN_IP_STATUS_FAIL	Command has not been accepted .
FLEXIO_LIN_IP_STATUS_TX_OK,Master	Header or Response successful transmission. Slave: Response successful transmission.
FLEXIO_LIN_IP_STATUS_TX_BUSY,Master	Ongoing transmission (Header or Response). Slave: Ongoing transmission response.
FLEXIO_LIN_IP_STATUS_TX_HEADER_ERR↩ OR,Master	Erroneous header transmission such as: Mismatch between sent and read back data or Physical bus error./
FLEXIO_LIN_IP_STATUS_TX_ERROR,Master	or Slave: Erroneous response transmission such as mismatch between sent and read back data, Physical bus error
FLEXIO_LIN_IP_STATUS_RX_OK,Master	or Slave: Reception of correct response.

Return values

FLEXIO_LIN_IP_STATUS_RX_BUSY,Master	or Slave: Ongoing reception. At least one response byte has been received, but the checksum byte has not been received.
FLEXIO_LIN_IP_STATUS_RX_ERROR,Master	or Slave: Erroneous response reception such as: - Framing error - Overrun error - Checksum error or Short response(timeout occurred).
FLEXIO_LIN_IP_STATUS_RX_NO_RESPONSE,Master	or Slave: No response byte has been received so far(timeout occurred).
FLEXIO_LIN_IP_STATUS_RX_HEADER_OK,Slave	Reception of correct header.
FLEXIO_LIN_IP_STATUS_RX_HEADER_BUSY,Slave	Ongoing header reception.
FLEXIO_LIN_IP_STATUS_RX_HEADER_ERROR,Slave	Erroneous header reception such as: Break field error, Sync field error, Identifier parity error, Framing error, timeout occurred or Physical bus error.
FLEXIO_LIN_IP_STATUS_OPERATIONAL,Normal	operation; there is no data has been sent or received after channel initialized or switched to IDLE from SLEEP.
FLEXIO_LIN_IP_STATUS_SLEEP	Sleep state operation.

6.1.6.8 Flexio_Lin_Ip_SendFrame()

```
Flexio_Lin_Ip_StatusType Flexio_Lin_Ip_SendFrame (
            const uint8 Channel,
            const Flexio_Lin_Ip_PduType * PduInfo )
```

Sends a LIN frame as master or a response as a slave.

Parameters

channel	LIN_FLEXIO channel number
PduInfo	Structure which contains information about the current frame which is to be transfered such as PID, Data Length, Type of checksum, Data buffer and Type of the response expected.

Returns

operation status:

Return values

FLEXIO_LIN_IP_STATUS_SUCCESS	: Command has not been accepted .
FLEXIO_LIN_IP_STATUS_BUSY	: Driver is busy with another transfer.
FLEXIO_LIN_IP_STATUS_ERROR	: Parameters such as Data Length and Pid are not correct or the current node is in sleep mode.

6.1.6.9 Flexio_Lin_Ip_TimerExpiredService()

```
void Flexio_Lin_Ip_TimerExpiredService (
            const uint8 Channel )
```

This is callback function for Timer Interrupt Handler. Users shall initialize a timer (for example FTM) and the time period in microseconds will be set by the driver via LinFlexioStartTimerNotification. In timer IRQ handler, call this function.

6.2 LIN Driver

6.2.1 Detailed Description

Data Structures

- struct Lin_43_LPUART_FLEXIO_ChannelConfigType

  LIN channel configuration type structure. *More...*
- struct Lin_43_LPUART_FLEXIO_ConfigType

  LIN driver configuration type structure. *More...*

Macros

- #define LIN_43_LPUART_FLEXIO_E_UNINIT

  API service used without module initialization.
- #define LIN_43_LPUART_FLEXIO_E_INVALID_CHANNEL

  API service used with an invalid or inactive channel parameter.
- #define LIN_43_LPUART_FLEXIO_E_INIT_FAILED

  API service called with invalid configuration pointer.
- #define LIN_43_LPUART_FLEXIO_E_STATE_TRANSITION

  Invalid state transition for the current state.
- #define LIN_43_LPUART_FLEXIO_E_PARAM_POINTER

  API service called with a NULL pointer.
- #define LIN_43_LPUART_FLEXIO_E_TIMEOUT

  Timeout caused by hardware error.
- #define LIN_43_LPUART_FLEXIO_E_PARAM_CONFIG

  API service called with the requested resource is not configured to be available on the current core.
- #define LIN_43_LPUART_FLEXIO_UNINIT

  LIN driver states.
- #define LIN_43_LPUART_FLEXIO_INIT

  LIN driver states.
- #define LIN_43_LPUART_FLEXIO_CH_SLEEP_PENDING

  LIN Channel states.
- #define LIN_43_LPUART_FLEXIO_CH_SLEEP_STATE

  LIN Channel states.
- #define LIN_43_LPUART_FLEXIO_CH_OPERATIONAL

  LIN Channel states.
- #define LIN_43_LPUART_FLEXIO_GETSTATUS_ID

  API service ID for Lin_43_LPUART_FLEXIO_GetStatus() function.
- #define LIN_43_LPUART_FLEXIO_GETVERSIONINFO_ID

  API service ID for Lin_43_LPUART_FLEXIO_GetVersionInfo() function.
- #define LIN_43_LPUART_FLEXIO_GOTOSLEEP_ID

  API service ID for Lin_43_LPUART_FLEXIO_GoToSleep() function.
- #define LIN_43_LPUART_FLEXIO_GOTOSLEEPINTERNAL_ID

API service ID for Lin_43_LPUART_FLEXIO_GoToSleepInternal() function.

- #define LIN_43_LPUART_FLEXIO_INIT_ID

  API service ID for Lin_43_LPUART_FLEXIO_Init() function.

- #define LIN_43_LPUART_FLEXIO_SENDFRAME_ID

  API service ID for Lin_43_LPUART_FLEXIO_SendFrame() function.

- #define LIN_43_LPUART_FLEXIO_WAKEUP_ID

  API service ID for Lin_43_LPUART_FLEXIO_WakeUp() function.

- #define LIN_43_LPUART_FLEXIO_WAKEUPINTERNAL_ID

  API service ID for Lin_43_LPUART_FLEXIO_WakeupInternal() function.

- #define LIN_43_LPUART_FLEXIO_CHECKWAKEUP_ID

  API service ID for Lin_43_LPUART_FLEXIO_CheckWakeup() function.

- #define LIN_43_LPUART_FLEXIO_TIMEOUT_ERROR

  Return code for timeout error.

Function Reference

- Std_ReturnType Lin_43_LPUART_FLEXIO_CheckWakeup (uint8 Channel)

  Validates for upper layers the wake up of LIN channel.

- void Lin_43_LPUART_FLEXIO_Init (const Lin_43_LPUART_FLEXIO_ConfigType *Config)

  Initializes the LIN module.

- Lin_StatusType Lin_43_LPUART_FLEXIO_GetStatus (uint8 Channel, const uint8 **Lin_43_LPUAR↩ T_FLEXIO_SduPtr)

  Gets the status of the LIN driver when Channel is operating.

- Std_ReturnType Lin_43_LPUART_FLEXIO_SendFrame (uint8 Channel, const Lin_PduType *PduInfo↩ Ptr)

  Sends a LIN frame.

- Std_ReturnType Lin_43_LPUART_FLEXIO_GoToSleep (uint8 Channel)

  Prepares and send a go-to-sleep-command frame on Channel.

- Std_ReturnType Lin_43_LPUART_FLEXIO_GoToSleepInternal (uint8 Channel)

  Same function as Lin_43_LPUART_FLEXIO_Ipw_GoToSleep() but without sending a go-to-sleep-command on the bus.

- Std_ReturnType Lin_43_LPUART_FLEXIO_Wakeup (uint8 Channel)

  Generates a wake up pulse.

- Std_ReturnType Lin_43_LPUART_FLEXIO_WakeupInternal (uint8 Channel)

  Wake up the LIN channel.

- void Lin_43_LPUART_FLEXIO_GetVersionInfo (Std_VersionInfoType *versioninfo)

  Returns the version information of this module.

6.2.2 Data Structure Documentation

6.2.2.1 struct Lin_43_LPUART_FLEXIO_ChannelConfigType

LIN channel configuration type structure.

This is the type of the external data structure containing the overall initialization data for one LIN Channel. A pointer to such a structure is provided to the LIN channel initialization routine for configuration of the LIN hardware channel.

Definition at line 143 of file Lin_43_LPUART_FLEXIO_Types.h.

Data Fields

uint8	LinChannelID	
const Lin_43_LPUART_FLEXIO_HwConfigType *	ChannelConfigPtr	Lin Channel ID. !<
uint32	ChannelPartitionID	LIN Hardware configuration pointer. !<
boolean	AllocatedPartition	LIN Channel Partition ID. !<

6.2.2.2 struct Lin_43_LPUART_FLEXIO_ConfigType

LIN driver configuration type structure.

This is the type of the pointer to the external data LIN Channels. A pointer of such a structure is provided to the LIN driver initialization routine for configuration of the LIN hardware channel.

Definition at line 163 of file Lin_43_LPUART_FLEXIO_Types.h.

Data Fields

- const Lin_43_LPUART_FLEXIO_ChannelConfigType * Lin_43_LPUART_FLEXIO_ChannelPtr [(2U)]

  Partition id is assigned for this configuration.

Field Documentation

Lin_43_LPUART_FLEXIO_ChannelPtr

```
const Lin_43_LPUART_FLEXIO_ChannelConfigType* Lin_43_LPUART_FLEXIO_ChannelPtr[(2U)]
```

Partition id is assigned for this configuration.

!<

Hardware channel.

Constant pointer of the constant external data structure containing the overall initialization data for all the configured LIN Channels.

Definition at line 172 of file Lin_43_LPUART_FLEXIO_Types.h.

6.2.3 Macro Definition Documentation

6.2.3.1 LIN_43_LPUART_FLEXIO_E_UNINIT

```
#define LIN_43_LPUART_FLEXIO_E_UNINIT
```

API service used without module initialization.

The LIN Driver module shall report the development error "LIN_43_LPUART_FLEXIO_E_UNINIT (0x00)", when the API Service is used without module initialization.

Definition at line 147 of file Lin_43_LPUART_FLEXIO.h.

6.2.3.2 LIN_43_LPUART_FLEXIO_E_INVALID_CHANNEL

```
#define LIN_43_LPUART_FLEXIO_E_INVALID_CHANNEL
```

API service used with an invalid or inactive channel parameter.

The LIN Driver module shall report the development error "LIN_43_LPUART_FLEXIO_E_INVALID_CHAN↩ NEL (0x02)", when API Service used with an invalid or inactive channel parameter.

Definition at line 157 of file Lin_43_LPUART_FLEXIO.h.

6.2.3.3 LIN_43_LPUART_FLEXIO_E_INIT_FAILED

```
#define LIN_43_LPUART_FLEXIO_E_INIT_FAILED
```

API service called with invalid configuration pointer.

The LIN Driver module shall report the development error "LIN_43_LPUART_FLEXIO_E_INIT_FAILE↩ D (0x03)", when API Service is called with invalid configuration pointer.

Definition at line 167 of file Lin_43_LPUART_FLEXIO.h.

6.2.3.4 LIN_43_LPUART_FLEXIO_E_STATE_TRANSITION

```
#define LIN_43_LPUART_FLEXIO_E_STATE_TRANSITION
```

Invalid state transition for the current state.

The LIN Driver module shall report the development error "LIN_43_LPUART_FLEXIO_E_STATE_TRANSI↩ TION (0x04)", when Invalid state transition occurs from the current state.

Definition at line 177 of file Lin_43_LPUART_FLEXIO.h.

6.2.3.5 LIN_43_LPUART_FLEXIO_E_PARAM_POINTER

```
#define LIN_43_LPUART_FLEXIO_E_PARAM_POINTER
```

API service called with a NULL pointer.

The LIN Driver module shall report the development error "LIN_43_LPUART_FLEXIO_E_PARAM_POINT↩ ER (0x05)", when API Service is called with a NULL pointer. In case of this error, the API service shall return immediately without any further action, beside reporting this development error.

Definition at line 189 of file Lin_43_LPUART_FLEXIO.h.

6.2.3.6 LIN_43_LPUART_FLEXIO_E_TIMEOUT

```
#define LIN_43_LPUART_FLEXIO_E_TIMEOUT
```

Timeout caused by hardware error.

The LIN Driver module shall report the development error "LIN_43_LPUART_FLEXIO_E_TIMEOUT (0x06)", when the error "Timeout caused by hardware error" has occurred and the reference LinDemEventParameterRefs/↩ LIN_E_TIMEOUT is not configured.

Definition at line 200 of file Lin_43_LPUART_FLEXIO.h.

6.2.3.7 LIN_43_LPUART_FLEXIO_E_PARAM_CONFIG

```
#define LIN_43_LPUART_FLEXIO_E_PARAM_CONFIG
```

API service called with the requested resource is not configured to be available on the current core.

The LIN will check upon each API call if the requested resource is configured to be available on the current core, and in case of error will return LIN_43_LPUART_FLEXIO_E_PARAM_CONFIG

Definition at line 209 of file Lin_43_LPUART_FLEXIO.h.

6.2.3.8 LIN_43_LPUART_FLEXIO_UNINIT

```
#define LIN_43_LPUART_FLEXIO_UNINIT
```

LIN driver states.

The state LIN_43_LPUART_FLEXIO_UNINIT means that the Lin module has not been initialized yet and cannot be used.

Definition at line 220 of file Lin_43_LPUART_FLEXIO.h.

6.2.3.9 LIN_43_LPUART_FLEXIO_INIT

```
#define LIN_43_LPUART_FLEXIO_INIT
```

LIN driver states.

The LIN_43_LPUART_FLEXIO_INIT state indicates that the LIN driver has been initialized, making each available channel ready for service.

Definition at line 229 of file Lin_43_LPUART_FLEXIO.h.

6.2.3.10 LIN_43_LPUART_FLEXIO_CH_SLEEP_PENDING

```
#define LIN_43_LPUART_FLEXIO_CH_SLEEP_PENDING
```

LIN Channel states.

go-to-sleep-command has been issued on the bus, LIN channel stay at this state until Lin_43_LPUART_FLEXIO_GetStatus() is called

Definition at line 238 of file Lin_43_LPUART_FLEXIO.h.

6.2.3.11 LIN_43_LPUART_FLEXIO_CH_SLEEP_STATE

```
#define LIN_43_LPUART_FLEXIO_CH_SLEEP_STATE
```

LIN Channel states.

The detection of a wake-up pulse is enabled. The LIN hardware is into a low power mode if such a mode is provided by the hardware.

Definition at line 248 of file Lin_43_LPUART_FLEXIO.h.

6.2.3.12 LIN_43_LPUART_FLEXIO_CH_OPERATIONAL

```
#define LIN_43_LPUART_FLEXIO_CH_OPERATIONAL
```

LIN Channel states.

The individual channel has been initialized (using at least one statically configured data set) and is able to participate in the LIN cluster.

Definition at line 258 of file Lin_43_LPUART_FLEXIO.h.

6.2.3.13 LIN_43_LPUART_FLEXIO_GETSTATUS_ID

```
#define LIN_43_LPUART_FLEXIO_GETSTATUS_ID
```

API service ID for Lin_43_LPUART_FLEXIO_GetStatus() function.

Parameters used when raising an error or exception.

Definition at line 268 of file Lin_43_LPUART_FLEXIO.h.

6.2.3.14 LIN_43_LPUART_FLEXIO_GETVERSIONINFO_ID

```
#define LIN_43_LPUART_FLEXIO_GETVERSIONINFO_ID
```

API service ID for Lin_43_LPUART_FLEXIO_GetVersionInfo() function.

Parameters used when raising an error or exception.

Definition at line 276 of file Lin_43_LPUART_FLEXIO.h.

6.2.3.15 LIN_43_LPUART_FLEXIO_GOTOSLEEP_ID

```
#define LIN_43_LPUART_FLEXIO_GOTOSLEEP_ID
```

API service ID for Lin_43_LPUART_FLEXIO_GoToSleep() function.

Parameters used when raising an error or exception.

Definition at line 284 of file Lin_43_LPUART_FLEXIO.h.

6.2.3.16 LIN_43_LPUART_FLEXIO_GOTOSLEEPINTERNAL_ID

```
#define LIN_43_LPUART_FLEXIO_GOTOSLEEPINTERNAL_ID
```

API service ID for Lin_43_LPUART_FLEXIO_GoToSleepInternal() function.

Parameters used when raising an error or exception.

Definition at line 292 of file Lin_43_LPUART_FLEXIO.h.

6.2.3.17 LIN_43_LPUART_FLEXIO_INIT_ID

```
#define LIN_43_LPUART_FLEXIO_INIT_ID
```

API service ID for Lin_43_LPUART_FLEXIO_Init() function.

Parameters used when raising an error or exception.

Definition at line 300 of file Lin_43_LPUART_FLEXIO.h.

6.2.3.18 LIN_43_LPUART_FLEXIO_SENDFRAME_ID

```
#define LIN_43_LPUART_FLEXIO_SENDFRAME_ID
```

API service ID for Lin_43_LPUART_FLEXIO_SendFrame() function.

Parameters used when raising an error or exception.

Definition at line 308 of file Lin_43_LPUART_FLEXIO.h.

6.2.3.19 LIN_43_LPUART_FLEXIO_WAKEUP_ID

```
#define LIN_43_LPUART_FLEXIO_WAKEUP_ID
```

API service ID for Lin_43_LPUART_FLEXIO_WakeUp() function.

Parameters used when raising an error or exception.

Definition at line 316 of file Lin_43_LPUART_FLEXIO.h.

6.2.3.20 LIN_43_LPUART_FLEXIO_WAKEUPINTERNAL_ID

```
#define LIN_43_LPUART_FLEXIO_WAKEUPINTERNAL_ID
```

API service ID for Lin_43_LPUART_FLEXIO_WakeupInternal() function.

Parameters used when raising an error or exception.

Definition at line 324 of file Lin_43_LPUART_FLEXIO.h.

6.2.3.21 LIN_43_LPUART_FLEXIO_CHECKWAKEUP_ID

```
#define LIN_43_LPUART_FLEXIO_CHECKWAKEUP_ID
```

API service ID for Lin_43_LPUART_FLEXIO_CheckWakeup() function.

Parameters used when raising an error or exception.

Definition at line 332 of file Lin_43_LPUART_FLEXIO.h.

6.2.3.22 LIN_43_LPUART_FLEXIO_TIMEOUT_ERROR

```
#define LIN_43_LPUART_FLEXIO_TIMEOUT_ERROR
```

Return code for timeout error.

Definition at line 119 of file Lin_43_LPUART_FLEXIO_Types.h.

6.2.4 Function Reference

6.2.4.1 Lin_43_LPUART_FLEXIO_CheckWakeup()

```
Std_ReturnType Lin_43_LPUART_FLEXIO_CheckWakeup (
            uint8 Channel )
```

Validates for upper layers the wake up of LIN channel.

This function identifies if the addressed LIN channel has been woken up by the LIN bus transceiver. It checks the wake up flag from the addressed LIN channel which must be in sleep mode and have the wake up signal.

Parameters

in	*Channel*	LIN channel to be addressed.

Return values

E_NOT_OK	If the LIN Channel is not valid or LIN driver is not initialized or the addressed LIN Channel is not in sleep state.
E_OK	Otherwise.

6.2.4.2 Lin_43_LPUART_FLEXIO_Init()

```
void Lin_43_LPUART_FLEXIO_Init (
            const Lin_43_LPUART_FLEXIO_ConfigType * Config )
```

Initializes the LIN module.

This function performs software initialization of LIN driver:

- Clears the shadow buffer of all available Lin channels
- Sets all the available channels to sleep mode and configures their state machine to LIN_43_LPUART_FL↩ EXIO_CH_SLEEP_STATE.

- Set driver state machine to LIN_43_LPUART_FLEXIO_INIT.

Parameters

in	*Config*	- Pointer to LIN driver configuration set.

Returns

void

Precondition

-

6.2.4.3 Lin_43_LPUART_FLEXIO_GetStatus()

```
Lin_StatusType Lin_43_LPUART_FLEXIO_GetStatus (
            uint8 Channel,
            const uint8 ** Lin_43_LPUART_FLEXIO_SduPtr )
```

Gets the status of the LIN driver when Channel is operating.

This function returns the state of the current transmission, reception or operation status. If the reception of a Slave response was successful then this service provides a pointer to the buffer where the data is stored.

Parameters

in	*Channel*	LIN channel to be addressed
out	*LinSduPtr*	pointer to pointer to a shadow buffer or memory mapped LIN Hardware receive buffer where the current SDU is stored

Returns

Lin_StatusType.

Return values

LIN_NOT_OK	Development or production error rised none of the below conditions.
LIN_TX_OK	Successful transmission.
LIN_TX_BUSY	Ongoing transmission of header or response.
LIN_TX_HEADER_ERROR	Error occurred during header transmission.
LIN_TX_ERROR	Error occurred during response transmission.
LIN_RX_OK	Reception of correct response.
LIN_RX_BUSY	Ongoing reception where at least one byte has been received.
LIN_RX_ERROR	Error occurred during reception.
LIN_RX_NO_REPONSE	No data byte has been received yet.
LIN_OPERATIONAL	Channel is ready for next header. transmission and no data are available.

Precondition

: This API is available only if at least one node is configured as MASTER.

6.2.4.4 Lin_43_LPUART_FLEXIO_SendFrame()

```
Std_ReturnType Lin_43_LPUART_FLEXIO_SendFrame (
            uint8 Channel,
            const Lin_PduType * PduInfoPtr )
```

Sends a LIN frame.

Sends a LIN header and a LIN response, if necessary. The direction of the frame response (master response, slave response, slave-to-slave communication) is provided by the PduInfoPtr.

Parameters

in	*Channel*	LIN channel to be addressed.
in	*PduInfoPtr*	pointer to PDU containing the PID, Checksum model, Response type, Dl and SDU data pointer.

Returns

Std_ReturnType.

Return values

E_NOT_OK	If the LIN Channel is not valid or LIN driver is not initialized or PduInfoPtr is NULL or a timeout occurs or LIN Channel is in sleep state.
E_OK	Otherwise.

Precondition

: Lin_43_LPUART_FLEXIO_Init function must be called before this API. This API is available only if at least one node is configured as MASTER.

6.2.4.5 Lin_43_LPUART_FLEXIO_GoToSleep()

```
Std_ReturnType Lin_43_LPUART_FLEXIO_GoToSleep (
            uint8 Channel )
```

Prepares and send a go-to-sleep-command frame on Channel.

This function stops any ongoing transmission and initiates the transmission of the sleep command (master command frame with ID = 0x3C and data = (0x00, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF).

Parameters

in	*Channel*	LIN channel to be addressed.

Returns

Std_ReturnType.

Return values

E_NOT_OK	In case of a timeout situation only.
E_OK	Otherwise.

Precondition

: Lin_43_LPUART_FLEXIO_Init function must be called before this API. This API is available only if at least one node is configured as MASTER.

6.2.4.6 Lin_43_LPUART_FLEXIO_GoToSleepInternal()

```
Std_ReturnType Lin_43_LPUART_FLEXIO_GoToSleepInternal (
            uint8 Channel )
```

Same function as Lin_43_LPUART_FLEXIO_Ipw_GoToSleep() but without sending a go-to-sleep-command on the bus.

This function stops any ongoing transmission and put the Channel in sleep mode (then LIN hardware enters a reduced power operation mode).

Parameters

in	*Channel*	LIN channel to be addressed.

Returns

Std_ReturnType.

Return values

E_NOT_OK	In case of a timeout situation only.
E_OK	Otherwise.

Precondition

: Lin_43_LPUART_FLEXIO_Init function must be called before this API.

6.2.4.7 Lin_43_LPUART_FLEXIO_Wakeup()

```
Std_ReturnType Lin_43_LPUART_FLEXIO_Wakeup (
            uint8 Channel )
```

Generates a wake up pulse.

This function shall sent a wake up signal to the LIN bus and put the LIN channel in LIN_43_LPUART_FLEXI↩ O_CH_OPERATIONAL state.

Parameters

in	*Channel*	LIN channel to be addressed.

Returns

Std_ReturnType.

Return values

E_NOT_OK	If the LIN driver is not in sleep state or LIN Channel is not valid or LIN driver is not initialized.
E_OK	Otherwise.

Precondition

: Lin_43_LPUART_FLEXIO_Init function must be called before this API.

6.2.4.8 Lin_43_LPUART_FLEXIO_WakeupInternal()

```
Std_ReturnType Lin_43_LPUART_FLEXIO_WakeupInternal (
            uint8 Channel )
```

Wake up the LIN channel.

This function shall put the LIN channel in LIN_43_LPUART_FLEXIO_CH_OPERATIONAL state without sending a wake up signal to the LIN bus

Parameters

in	*Channel*	LIN channel to be addressed.

Returns

Std_ReturnType.

Return values

E_NOT_OK	If the LIN driver is not in sleep state or LIN Channel is not valid or LIN driver is not initialized.
E_OK	Otherwise.

Precondition

: Lin_43_LPUART_FLEXIO_Init function must be called before this API.

6.2.4.9 Lin_43_LPUART_FLEXIO_GetVersionInfo()

```
void Lin_43_LPUART_FLEXIO_GetVersionInfo (
            Std_VersionInfoType * versioninfo )
```

Returns the version information of this module.

The version information includes:

```
- Two bytes for the Vendor ID
- Two bytes for the Module ID
- One byte  for the Instance ID
- Three bytes version number. The numbering shall be vendor
  specific: it consists of:

   - The major, the minor and the patch version number of
     the module;
   - The AUTOSAR specification version number shall not
     be included. The AUTOSAR specification version number is
     checked during compile time and therefore not required
     in this API.
```

Parameters

in,out	*versioninfo*	Pointer for storing the version information of this module.

Returns

void.

6.3 LIN

6.3.1 Detailed Description

Macros

- #define LIN_43_LPUART_FLEXIO_SETCLOCKMODE_ID

  API service ID for Lin_43_LPUART_FLEXIO_SetClockMode() function.

Enum Reference

- enum Lin_43_LPUART_FLEXIO_ClockModesType

  Clock modes.

Function Reference

- Std_ReturnType Lin_43_LPUART_FLEXIO_SetClockMode (Lin_43_LPUART_FLEXIO_ClockModesType ClockMode)

  Lin_43_LPUART_FLEXIO_SetClockMode.

6.3.2 Macro Definition Documentation

6.3.2.1 LIN_43_LPUART_FLEXIO_SETCLOCKMODE_ID

```
#define LIN_43_LPUART_FLEXIO_SETCLOCKMODE_ID
```

API service ID for Lin_43_LPUART_FLEXIO_SetClockMode() function.

Parameters used when raising an error or exception.

Definition at line 94 of file Lin_43_LPUART_FLEXIO_ASRExt.h.

6.3.3 Enum Reference

6.3.3.1 Lin_43_LPUART_FLEXIO_ClockModesType

```
enum Lin_43_LPUART_FLEXIO_ClockModesType
```

Clock modes.

Precondition

LIN_43_LPUART_FLEXIO_DUAL_CLOCK_MODE must be defined and its value must be STD_ON.

Note

Possible clock modes: LIN_43_LPUART_FLEXIO_NORMAL (normal execution mode), LIN_43_LPUA↩ RT_FLEXIO_ALTERNATE (low power mode).

Enumerator

LIN_43_LPUART_FLEXIO_NORMAL	LIN_43_LPUART_FLEXIO_NORMAL mode.
LIN_43_LPUART_FLEXIO_ALTERNATE	LIN_43_LPUART_FLEXIO_ALTERNATE mode.

Definition at line 110 of file Lin_43_LPUART_FLEXIO_ASRExt.h.

6.3.4 Function Reference

6.3.4.1 Lin_43_LPUART_FLEXIO_SetClockMode()

```
Std_ReturnType Lin_43_LPUART_FLEXIO_SetClockMode (
            Lin_43_LPUART_FLEXIO_ClockModesType ClockMode )
```

Lin_43_LPUART_FLEXIO_SetClockMode.

Switch the clock mode to the alternate clock mode on all the Lin channels.

Parameters

in	*ClockMode*	New clock mode.

Returns

The result of the switching clock operation.

Return values

E_OK	The switching operation was OK.
E_NOT_OK	The switching operation has failed caused by the wrong driver state.

Switch the clock mode to the alternate clock mode on all the Lin channels.

Precondition

LIN_43_LPUART_FLEXIO_DUAL_CLOCK_MODE must be defined and its value must be STD_ON.

Note

Switch the clock mode to the alternate clock mode on all the Lin channels.

6.4 LIN_43_LPUART_FLEXIO_IPW

6.4.1 Detailed Description

Enum Reference

- enum Lin_43_LPUART_FLEXIO_Ipw_HwChannelType
- enum Lin_43_LPUART_FLEXIO_Ipw_NodeType

  Definition of the LIN node type of the current channel.

6.4.2 Enum Reference

6.4.2.1 Lin_43_LPUART_FLEXIO_Ipw_HwChannelType

enum Lin_43_LPUART_FLEXIO_Ipw_HwChannelType

Enumerator

LIN_43_LPUART_FLEXIO_LPUART_CHANNEL	This is used for LPUART channels.
LIN_43_LPUART_FLEXIO_FLEXIO_CHANNEL	This is used for FLEXIO channels.

Definition at line 149 of file Lin_43_LPUART_FLEXIO_Ipw_Types.h.

6.4.2.2 Lin_43_LPUART_FLEXIO_Ipw_NodeType

enum Lin_43_LPUART_FLEXIO_Ipw_NodeType

Definition of the LIN node type of the current channel.

This type enumerates the node types for a LIN channel.

Enumerator

LIN_43_LPUART_FLEXIO_MASTER_NODE	This is used for Master node.
LIN_43_LPUART_FLEXIO_SLAVE_NODE	This is used for Slave node.

Definition at line 162 of file Lin_43_LPUART_FLEXIO_Ipw_Types.h.

6.5 Lpuart Lin IPL

6.5.1 Detailed Description

Data Structures

- struct Lpuart_Lin_Ip_PduType

  Define type for an structure containing information regarding the LIN Frame . More...

- struct Lpuart_Lin_Ip_StateStructType

  Runtime state of the LIN driver. More...

- struct Lpuart_Lin_Ip_UserConfigType

  User configuration structure of the LIN driver. More...

Macros

- #define LPUART_LIN_IP_SLAVE

  Macro that specifies usage of a SLAVE note.

- #define LPUART_LIN_IP_MASTER

  Macro that specifies usage of a SLAVE note.

- #define LPUART_LIN_IP_BREAK_CHAR_10_BIT_MINIMUM_U8

  Macro LPUART break char length 10 bit times.

- #define LPUART_LIN_IP_BREAK_CHAR_13_BIT_MINIMUM_U8

  Macro LPUART break char length 10 bit times.

Types Reference

- typedef void(* Lpuart_Lin_Ip_CallbackType) (const uint8 Instance, const Lpuart_Lin_Ip_StateStructType *LinState)

  LIN Driver callback function type.

Enum Reference

- enum Lpuart_Lin_Ip_EventIdType

  Enum containing the events related to a ID.

- enum Lpuart_Lin_Ip_NodeStateType

  Define type for an enumerating LIN Node state.

- enum Lpuart_Lin_Ip_StatusType

  LPUART status type.

- enum Lpuart_Lin_Ip_FrameCsModelType

  Define type for checksum type of the frame.

- enum Lpuart_Lin_Ip_FrameResponseType

  Define type for Response type of the frame.

- enum Lpuart_Lin_Ip_TransferStatusType

  Description: This type defines a range of transfer status.

Function Reference

- Lpuart_Lin_Ip_StatusType Lpuart_Lin_Ip_Init (const uint8 Instance, const Lpuart_Lin_Ip_UserConfigType *UserConfig)

  Initializes an LIN_LPUART instance for LIN Network.

- Lpuart_Lin_Ip_StatusType Lpuart_Lin_Ip_Deinit (const uint8 Instance)

  Shuts down the LIN_LPUART by disabling interrupts and transmitter/receiver.

- Lpuart_Lin_Ip_StatusType Lpuart_Lin_Ip_SendFrame (const uint8 Instance, const Lpuart_Lin_Ip_PduType *PduInfo)

  Sends frame out through the LIN_LPUART module using non-blocking method.

- Lpuart_Lin_Ip_StatusType Lpuart_Lin_Ip_AbortTransferData (const uint8 Instance)

  Aborts an on-going non-blocking transmission/reception.

- Lpuart_Lin_Ip_StatusType Lpuart_Lin_Ip_GoToSleepMode (const uint8 Instance)

  This function puts current node to sleep mode.

- void Lpuart_Lin_Ip_GoToIdleState (const uint8 Instance)

  Puts current LIN node to Idle state This function changes current node state to LIN_NODE_STATE_IDLE.

- Lpuart_Lin_Ip_StatusType Lpuart_Lin_Ip_SendWakeupSignal (const uint8 Instance)

  Sends a wakeup signal through the LIN_LPUART interface.

- Lpuart_Lin_Ip_TransferStatusType Lpuart_Lin_Ip_GetStatus (const uint8 Instance, const uint8 **Buffer)

  Returns the status of an on going transmission.

- void Lpuart_Lin_Ip_IRQHandler (const uint8 Instance)

  This is callback function for Timer Interrupt Handler. Users shall initialize a timer (for example FTM) and the time period in microseconds will be set by the driver via LinLpuartStartTimerNotification. In timer IRQ handler, call this function.

6.5.2 Data Structure Documentation

6.5.2.1 struct Lpuart_Lin_Ip_PduType

Define type for an structure containing information regarding the LIN Frame .

Definition at line 220 of file Lpuart_Lin_Ip_Types.h.

Data Fields

- uint8 Pid

  LIN frame identifier.

- Lpuart_Lin_Ip_FrameCsModelType Cs

  Checksum model type.

- Lpuart_Lin_Ip_FrameResponseType Drc

  Response type.

- uint8 Dl

  Data length.

- uint8 * SduPtr

  Pointer to Sdu.

Field Documentation

Pid

`uint8 Pid`

LIN frame identifier.

Definition at line 222 of file Lpuart_Lin_Ip_Types.h.

Cs

`Lpuart_Lin_Ip_FrameCsModelType Cs`

Checksum model type.

Definition at line 223 of file Lpuart_Lin_Ip_Types.h.

Drc

`Lpuart_Lin_Ip_FrameResponseType Drc`

Response type.

Definition at line 224 of file Lpuart_Lin_Ip_Types.h.

Dl

`uint8 Dl`

Data length.

Definition at line 225 of file Lpuart_Lin_Ip_Types.h.

SduPtr

`uint8* SduPtr`

Pointer to Sdu.

Definition at line 226 of file Lpuart_Lin_Ip_Types.h.

6.5.2.2 struct Lpuart_Lin_Ip_StateStructType

Runtime state of the LIN driver.

Definition at line 258 of file Lpuart_Lin_Ip_Types.h.

Data Fields

const uint8 *	TxBuff	The buffer of data being sent.
uint8 *	RxBuff	The buffer of received data.
uint8	CntByte	To count number of bytes already transmitted or received.

Data Fields

volatile uint8	TxSize	The remaining number of bytes to be transmitted.
volatile uint8	RxSize	The remaining number of bytes to be received.
uint8	Checksum	Checksum byte.
volatile boolean	IsBusBusy	True if there are data, frame headers being transferred on bus.
uint8	CurrentPid	Current PID.
volatile Lpuart_Lin_Ip_EventIdType	CurrentEventId	Current ID Event.
volatile Lpuart_Lin_Ip_NodeStateType	CurrentNodeState	Current Node state.
uint32	LinSourceClockFreq	Frequency of the source clock for LIN.
volatile Lpuart_Lin_Ip_NodeStateType	PreviousNodeState	Store previous node state when set Lin channel to idle for further processing.

6.5.2.3 struct Lpuart_Lin_Ip_UserConfigType

User configuration structure of the LIN driver.

Definition at line 297 of file Lpuart_Lin_Ip_Types.h.

Data Fields

- uint8 Instance

  Hardware Instance.
- uint32 BaudRateDivisor

  Baudrate divider to be configured in hardware.
- uint32 OverSamplingRatio

  baudrate of LIN Hardware Interface to configure
- boolean NodeFunction

  Node function as Master or Slave.
- uint8 BreakLength

  Length of break character will be transmitted.
- uint8 BreakLengthDetect

  Length of break character can be detected.
- Lpuart_Lin_Ip_CallbackType Callback

  Callback function to invoke after receiving a byte or transmitting a byte.
- uint8 WakeupByte

  Byte will be sent to generate wakeup pulse [400us->4ms].
- uint32 ChannelClock

  Channel clock.

Field Documentation

Instance

`uint8 Instance`

Hardware Instance.

Definition at line 299 of file Lpuart_Lin_Ip_Types.h.

BaudRateDivisor

`uint32 BaudRateDivisor`

Baudrate divider to be configured in hardware.

Definition at line 300 of file Lpuart_Lin_Ip_Types.h.

OverSamplingRatio

`uint32 OverSamplingRatio`

baudrate of LIN Hardware Interface to configure

Definition at line 301 of file Lpuart_Lin_Ip_Types.h.

NodeFunction

`boolean NodeFunction`

Node function as Master or Slave.

Definition at line 302 of file Lpuart_Lin_Ip_Types.h.

BreakLength

`uint8 BreakLength`

Length of break character will be transmitted.

Definition at line 303 of file Lpuart_Lin_Ip_Types.h.

BreakLengthDetect

`uint8 BreakLengthDetect`

Length of break character can be detected.

Definition at line 304 of file Lpuart_Lin_Ip_Types.h.

Callback

`Lpuart_Lin_Ip_CallbackType Callback`

Callback function to invoke after receiving a byte or transmitting a byte.

Definition at line 308 of file Lpuart_Lin_Ip_Types.h.

WakeupByte

`uint8 WakeupByte`

Byte will be sent to generate wakeup pulse [400us->4ms].

Definition at line 316 of file Lpuart_Lin_Ip_Types.h.

ChannelClock

`uint32 ChannelClock`

Channel clock.

Definition at line 317 of file Lpuart_Lin_Ip_Types.h.

6.5.3 Macro Definition Documentation

6.5.3.1 LPUART_LIN_IP_SLAVE

`#define LPUART_LIN_IP_SLAVE`

Macro that specifies usage of a SLAVE note.

This macro is used in user configuration structure to set the node type as SLAVE.

Definition at line 95 of file Lpuart_Lin_Ip_Types.h.

6.5.3.2 LPUART_LIN_IP_MASTER

`#define LPUART_LIN_IP_MASTER`

Macro that specifies usage of a SLAVE note.

This macro is used in user configuration structure to set the node type as MASTER.

Definition at line 103 of file Lpuart_Lin_Ip_Types.h.

6.5.3.3 LPUART_LIN_IP_BREAK_CHAR_10_BIT_MINIMUM_U8

`#define LPUART_LIN_IP_BREAK_CHAR_10_BIT_MINIMUM_U8`

Macro LPUART break char length 10 bit times.

LPUART break char length 10 bit times (if M = 0, SBNS = 0) or 11 (if M = 1, SBNS = 0 or M = 0, SBNS = 1) or 12 (if M = 1, SBNS = 1 or M10 = 1, SNBS = 0) or 13 (if M10 = 1, SNBS = 1)

Definition at line 112 of file Lpuart_Lin_Ip_Types.h.

6.5.3.4 LPUART_LIN_IP_BREAK_CHAR_13_BIT_MINIMUM_U8

```
#define LPUART_LIN_IP_BREAK_CHAR_13_BIT_MINIMUM_U8
```

Macro LPUART break char length 10 bit times.

LPUART break char length 13 bit times (if M = 0, SBNS = 0 or M10 = 0, SBNS = 1) or 14 (if M = 1, SBNS = 0 or M = 1, SBNS = 1) or 15 (if M10 = 1, SBNS = 1 or M10 = 1, SNBS = 0)

Definition at line 121 of file Lpuart_Lin_Ip_Types.h.

6.5.4 Types Reference

6.5.4.1 Lpuart_Lin_Ip_CallbackType

```
typedef void(* Lpuart_Lin_Ip_CallbackType) (const uint8 Instance, const Lpuart_Lin_Ip_StateStructType
*LinState)
```

LIN Driver callback function type.

Definition at line 288 of file Lpuart_Lin_Ip_Types.h.

6.5.5 Enum Reference

6.5.5.1 Lpuart_Lin_Ip_EventIdType

```
enum Lpuart_Lin_Ip_EventIdType
```

Enum containing the events related to a ID.

This enum defines types for an enumerating event related to an Identifier.

Enumerator

LPUART_LIN_IP_NO_EVENT	No event.
LPUART_LIN_IP_WAKEUP_SIGNAL	Wakeup signal detected.
LPUART_LIN_IP_BAUDRATE_ADJUSTED	Baudrate adjusted.
LPUART_LIN_IP_RECV_BREAK_FIELD_OK	Break field received.
LPUART_LIN_IP_SYNC_ERROR	Sync byte received with errors.
LPUART_LIN_IP_SEND_HEADER_OK	Sync byte received ok.
LPUART_LIN_IP_RECV_HEADER_OK	PID byte received ok.
LPUART_LIN_IP_PID_ERROR	PID byte received with errors.
LPUART_LIN_IP_FRAME_ERROR	Frame transfer has errors.
LPUART_LIN_IP_READBACK_ERROR	Readback error.
LPUART_LIN_IP_CHECKSUM_ERROR_EVENT	Checksum error.
LPUART_LIN_IP_TX_COMPLETED	Tx completed.
LPUART_LIN_IP_RX_COMPLETED	Rx completed.
LPUART_LIN_IP_RX_OVERRUN_ERROR	Rx overrun error.
LPUART_LIN_IP_TIMEOUT_ERROR	Timeout error.

Definition at line 135 of file Lpuart_Lin_Ip_Types.h.

6.5.5.2 Lpuart_Lin_Ip_NodeStateType

enum Lpuart_Lin_Ip_NodeStateType

Define type for an enumerating LIN Node state.

Enumerator

LPUART_LIN_IP_NODE_STATE_UNINIT	Uninitialized state.
LPUART_LIN_IP_NODE_STATE_SLEEP_MODE	Sleep mode state.
LPUART_LIN_IP_NODE_STATE_IDLE	Idle state.
LPUART_LIN_IP_NODE_STATE_SEND_BREAK_FIELD	Send break field state.
LPUART_LIN_IP_NODE_STATE_SEND_SYNC	Send the synchronization byte state.
LPUART_LIN_IP_NODE_STATE_RECV_SYNC	Receive the synchronization byte state.
LPUART_LIN_IP_NODE_STATE_SEND_PID	Send PID state.
LPUART_LIN_IP_NODE_STATE_RECV_PID	Receive PID state.
LPUART_LIN_IP_NODE_STATE_RECV_DATA	Receive data state.
LPUART_LIN_IP_NODE_STATE_RECV_DATA_COMPLETED	Receive data completed state.
LPUART_LIN_IP_NODE_STATE_SEND_DATA	Send data state.
LPUART_LIN_IP_NODE_STATE_SEND_DATA_COMPLETED	Send data completed state.

Definition at line 161 of file Lpuart_Lin_Ip_Types.h.

6.5.5.3 Lpuart_Lin_Ip_StatusType

enum Lpuart_Lin_Ip_StatusType

LPUART status type.

Definition at line 181 of file Lpuart_Lin_Ip_Types.h.

6.5.5.4 Lpuart_Lin_Ip_FrameCsModelType

enum Lpuart_Lin_Ip_FrameCsModelType

Define type for checksum type of the frame.

Enumerator

LPUART_LIN_IP_ENHANCED_CS	Enhanced CheckSum model.
LPUART_LIN_IP_CLASSIC_CS	Classic CheckSum model.

Definition at line 192 of file Lpuart_Lin_Ip_Types.h.

6.5.5.5 Lpuart_Lin_Ip_FrameResponseType

enum Lpuart_Lin_Ip_FrameResponseType

Define type for Response type of the frame.

Enumerator

LPUART_LIN_IP_FRAMERESPONSE_TX	Response is generated from this node.
LPUART_LIN_IP_FRAMERESPONSE_RX	Response is generated from another node and is relevant for this node.
LPUART_LIN_IP_FRAMERESPONSE_IGNORE	Response is generated from another node and is irrelevant for this node.

Definition at line 202 of file Lpuart_Lin_Ip_Types.h.

6.5.5.6 Lpuart_Lin_Ip_TransferStatusType

enum Lpuart_Lin_Ip_TransferStatusType

Description: This type defines a range of transfer status.

Enumerator

LPUART_LIN_IP_STATUS_FAIL	Command has not been accepted.
LPUART_LIN_IP_STATUS_TX_OK	Transmission successfully.
LPUART_LIN_IP_STATUS_TX_BUSY	Transmission is ongoing.
LPUART_LIN_IP_STATUS_TX_HEADER_ERROR	Erroneous header transmission.
LPUART_LIN_IP_STATUS_TX_ERROR	Erroneous response transmission.
LPUART_LIN_IP_STATUS_RX_OK	Reception of correct response.
LPUART_LIN_IP_STATUS_RX_BUSY	Ongoing reception. At least one response byte has been received.
LPUART_LIN_IP_STATUS_RX_ERROR	Erroneous response reception.
LPUART_LIN_IP_STATUS_RX_NO_RESPONSE	No response byte has been received so far(timeout occurred).
LPUART_LIN_IP_STATUS_RX_HEADER_OK	Slave received a correct header.
LPUART_LIN_IP_STATUS_RX_HEADER_BUSY	Slave is receiving header.
LPUART_LIN_IP_STATUS_RX_HEADER_ERROR	Erroneous header reception of slave.
LPUART_LIN_IP_STATUS_OPERATIONAL	Normal operation.
LPUART_LIN_IP_STATUS_SLEEP	Sleep state operation.

Definition at line 233 of file Lpuart_Lin_Ip_Types.h.

6.5.6 Function Reference

6.5.6.1 Lpuart_Lin_Ip_Init()

```
Lpuart_Lin_Ip_StatusType Lpuart_Lin_Ip_Init (
            const uint8 Instance,
            const Lpuart_Lin_Ip_UserConfigType * UserConfig )
```

Initializes an LIN_LPUART instance for LIN Network.

The caller provides memory for the driver state structures during initialization. The user must select the LIN_LPU↩ ART clock source in the application to initialize the LIN_LPUART. This function initializes a LPUART instance for operation. This function will initialize the run-time state structure to keep track of the on-going transfers, initialize the module to user defined settings and default settings, set break field length to be 13 bit times minimum, enable the break detect interrupt, Rx complete interrupt, frame error detect interrupt, and enable the LPUART module transmitter and receiver

Parameters

Instance	LIN_LPUART instance number
UserConfig	user configuration structure of type #lin_user_config_t

Returns

LPUART_LIN_IP_STATUS_SUCCESS - Initialization command has been accepted. LPUART_LIN_IP↩ _STATUS_ERROR - Initialization command has not been accepted.

6.5.6.2 Lpuart_Lin_Ip_Deinit()

```
Lpuart_Lin_Ip_StatusType Lpuart_Lin_Ip_Deinit (
            const uint8 Instance )
```

Shuts down the LIN_LPUART by disabling interrupts and transmitter/receiver.

Parameters

Instance	LIN_LPUART instance number

Returns

LPUART_LIN_IP_STATUS_SUCCESS - De-initialization command has been accepted. LPUART_LIN↩ _IP_STATUS_ERROR - De-initialization command has not been accepted. *

6.5.6.3 Lpuart_Lin_Ip_SendFrame()

```
Lpuart_Lin_Ip_StatusType Lpuart_Lin_Ip_SendFrame (
            const uint8 Instance,
            const Lpuart_Lin_Ip_PduType * PduInfo )
```

Sends frame out through the LIN_LPUART module using non-blocking method.

This enables an a-sync method for transmitting lin frame. Non-blocking means that the function returns immediately. The application has to get the transferring status to know when the frame is complete. This function will calculate the checksum byte and send it with the frame data. If txSize is equal to 0 or greater than 8 or node's current state is in SLEEP mode then the function will return LPUART_LIN_IP_STATUS_ERROR. If isBusBusy is currently true then the function will return LPUART_LIN_IP_STATUS_BUSY.

Parameters

in	*Instance*	LIN_LPUART instance number
in	*PduInfo*	PDU containing the ID, Checksum model, Response type, Data Length and SDU data pointer.

Returns

operation status:

Return values

LPUART_LIN_IP_STATUS_SUCCESS	- Send command has been accepted.
LPUART_LIN_IP_STATUS_BUSY	- Operation failed due to hardware instance busy.
LPUART_LIN_IP_STATUS_ERROR	- Send command has not been accepted.

6.5.6.4 Lpuart_Lin_Ip_AbortTransferData()

```
Lpuart_Lin_Ip_StatusType Lpuart_Lin_Ip_AbortTransferData (
            const uint8 Instance )
```

Aborts an on-going non-blocking transmission/reception.

While performing a non-blocking transferring data, users can call this function to terminate immediately the transferring.

Parameters

Instance	LIN_LPUART instance number

Returns

Lpuart_Lin_Ip_StatusType

6.5.6.5 Lpuart_Lin_Ip_GoToSleepMode()

```
Lpuart_Lin_Ip_StatusType Lpuart_Lin_Ip_GoToSleepMode (
            const uint8 Instance )
```

This function puts current node to sleep mode.

This function changes current node state to LIN_NODE_STATE_SLEEP_MODE

Parameters

Instance	LIN_LPUART instance number

Returns

function always return LPUART_LIN_IP_STATUS_SUCCESS

6.5.6.6 Lpuart_Lin_Ip_GoToIdleState()

```
void Lpuart_Lin_Ip_GoToIdleState (
            const uint8 Instance )
```

Puts current LIN node to Idle state This function changes current node state to LIN_NODE_STATE_IDLE.

Parameters

Instance	LIN_LPUART instance number

Returns

function always return LPUART_LIN_IP_STATUS_SUCCESS

6.5.6.7 Lpuart_Lin_Ip_SendWakeupSignal()

```
Lpuart_Lin_Ip_StatusType Lpuart_Lin_Ip_SendWakeupSignal (
            const uint8 Instance )
```

Sends a wakeup signal through the LIN_LPUART interface.

Parameters

Instance	LIN_LPUART instance number

Returns

operation status:

Return values

LPUART_LIN_IP_STATUS_SUCCESS	: Command has been accepted.
LPUART_LIN_IP_STATUS_ERROR	: Command has not been accepted, error occurred.

6.5.6.8 Lpuart_Lin_Ip_GetStatus()

```
Lpuart_Lin_Ip_TransferStatusType Lpuart_Lin_Ip_GetStatus (
            const uint8 Instance,
            const uint8 ** Buffer )
```

Returns the status of an on going transmission.

This function returns the status of the current non-blocking transfer. If a response reception has been successfully received, Buffer will be referenced to receive buffer.

Parameters

channel	LIN_LPUART instance number.
Buffer	Pointer to the received data buffer.

Returns

operation status: LPUART_Lin_Ip_TransferStatusType

Return values

LPUART_LIN_IP_STATUS_FAIL	Command has not been accepted .
LPUART_LIN_IP_STATUS_TX_OK,Master	Header or Response successful transmission. Slave: Response successful transmission.
LPUART_LIN_IP_STATUS_TX_BUSY,Master	Ongoing transmission (Header or Response). Slave: Ongoing transmission response.
LPUART_LIN_IP_STATUS_TX_HEADER_ER↩ ROR,Master	Erroneous header transmission such as: Mismatch between sent and read back data or Physical bus error./
LPUART_LIN_IP_STATUS_TX_ERROR,Master	or Slave: Erroneous response transmission such as mismatch between sent and read back data, Physical bus error
LPUART_LIN_IP_STATUS_RX_OK,Master	or Slave: Reception of correct response.
LPUART_LIN_IP_STATUS_RX_BUSY,Master	or Slave: Ongoing reception. At least one response byte has been received, but the checksum byte has not been received.
LPUART_LIN_IP_STATUS_RX_ERROR,Master	or Slave: Erroneous response reception such as: - Framing error - Overrun error - Checksum error or Short response(timeout occurred).
LPUART_LIN_IP_STATUS_RX_NO_RESPON↩ SE,Master	or Slave: No response byte has been received so far(timeout occurred).
LPUART_LIN_IP_STATUS_RX_HEADER_↩ OK,Slave	Reception of correct header.
LPUART_LIN_IP_STATUS_RX_HEADER_BU↩ SY,Slave	Ongoing header reception.
LPUART_LIN_IP_STATUS_RX_HEADER_ER↩ ROR,Slave	Erroneous header reception such as: Break field error, Sync field error, Identifier parity error, Framing error, timeout occurred or Physical bus error.
LPUART_LIN_IP_STATUS_OPERATION↩ AL,Normal	operation; there is no data has been sent or received after channel initialized or switched to IDLE from SLEEP.
LPUART_LIN_IP_STATUS_SLEEP	Sleep state operation.

6.5.6.9 Lpuart_Lin_Ip_IRQHandler()

```
void Lpuart_Lin_Ip_IRQHandler (
            const uint8 Instance )
```

This is callback function for Timer Interrupt Handler. Users shall initialize a timer (for example FTM) and the time

period in microseconds will be set by the driver via LinLpuartStartTimerNotification. In timer IRQ handler, call this function.

Parameters

Instance	LIN_LPUART instance number

Returns

None

LIN_LPUART interrupt handler for RX_TX and Error interrupts.

Parameters

Instance	LIN_LPUART instance number

Returns

void

How to Reach Us:

Home Page:
nxp.com

Web Support:
nxp.com/support

Limited warranty and liability - Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors.
In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.
Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes - NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use in automotive applications - This NXP product has been qualified for use in automotive applications. If this product is used by customer in the development of, or for incorporation into, products or services (a) used in safety critical applications or (b) in which failure could lead to death, personal injury, or severe physical or environmental damage (such products and services hereinafter referred to as "Critical Applications"), then customer makes the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, safety, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP. As such, customer assumes all risk related to use of any products in Critical Applications and NXP and its suppliers shall not be liable for any such use by customer. Accordingly, customer will indemnify and hold NXP harmless from any claims, liabilities, damages and associated costs and expenses (including attorneys' fees) that NXP may incur related to customer's incorporation of any product in a Critical Application.

Applications - Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.
Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.
NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Terms and conditions of commercial sale - NXP Semiconductors products are sold subject to the general terms and conditions of commercial sale, as published at http://www.nxp.com/profile/terms, unless otherwise agreed in a valid written individual agreement. In case an individual agreement is concluded only the terms and conditions of the respective agreement shall apply. NXP Semiconductors hereby expressly objects to applying the customer's general terms and conditions with regard to the purchase of NXP Semiconductors products by customer.

Export control - This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Translations — A non-English (translated) version of a document, including the legal information in that document, is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

Security — Customer understands that all NXP products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Customer is responsible for the design and operation of its applications and products throughout their lifecycles to reduce the effect of these vulnerabilities on customer's applications and products. Customer's responsibility also extends to other open and/or proprietary technologies supported by NXP products for use in customer's applications. NXP accepts no liability for any vulnerability. Customer should regularly check security updates from NXP and follow up appropriately.
Customer shall select products with security features that best meet rules, regulations, and standards of the intended application and make the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP.
NXP has a Product Security Incident Response Team (PSIRT) (reachable at mailto:PSIRT@nxp.com) that manages the investigation, reporting, and solution release to security vulnerabilities of NXP products.

NXP B.V. - NXP B.V. is not an operating company and it does not distribute or sell products.